Bases de Datos

Clase 13: ORM – NoSQL y full-text search & retrieval

Lo que hemos visto

- ✓ Lo que es un modelo Relacional
- ✓ Hacer consultas en Álgebra Relacional
- ✓ Modelar un problema usando el modelo E/R
- ✓ Transformar un MER a un esquema relacional
- ✓ Identificar dependencias Funcionales Anomalías
- ✓ 1,2,3 y BC Formas Normales → Proyecto E1
- ✓ SQL → Proyecto E2, E3
- ✓ Vistas, Triggers y Stored Procedures
- ✓ Transacciones, ACID, Schedules y Locks → Proyecto E4
- ✓ Recuperación ante fallas
- ✓ Índices
- ✓ Algoritmos… Select, Join Hasta aquí bases de datos Relacionales

Hoy: NoSQL

Datos según su grado de estructuración

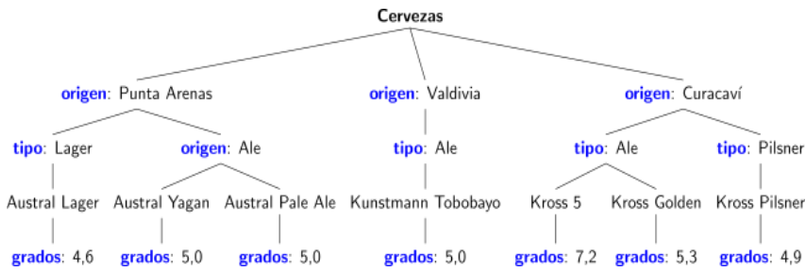
Relacional
(SQL, CSV, ...)

Cervezas

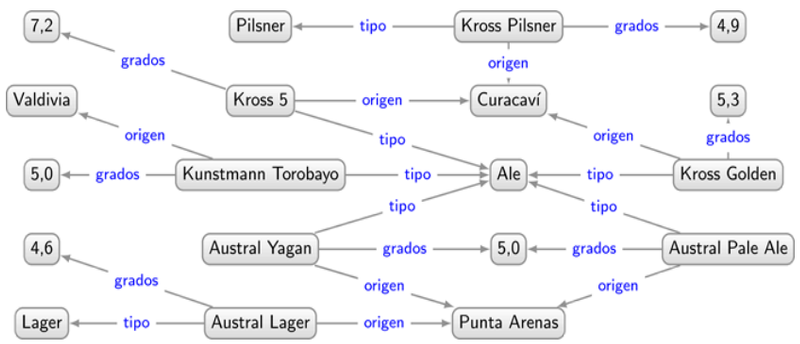
nombre	tipo	grados	ciudad-origen
Austral Lager	Lager	4,6	Punta Arenas
Austral Yagan	Ale	5,0	Punta Arenas
Austral Pale Ale	Ale	5,0	Punta Arenas
Kuntsmann Torobayo	Ale	5,0	Valdivia
Kross 5	Ale	7,2	Curacaví
Kross Golden	Ale	5,3	Curacaví
Kross Pilsner	Pilsner	4,9	Curacaví

Estructurados

Árboles
(XML, JSON, ...)



Grafos
(RDF, Prop. Gs, ...)



Semiestructurados

Texto Enriquecido
(HTML, Word, ...)

```
<ul>
  <li>Austral Lager [Lager] 4,6% de Punta Arenas</li>
  <li>Austral Yagan [Ale] 5% de Punta Arenas</li>
  <li>Austral Pale Ale [Ale] 5% de Punta Arenas</li>
  <li>Kunstsmann Torobayo [Ale] 5% de Valdivia</li>
  <li>Kross 5 [Ale] 7,2% de Curacaví</li>
  <li>Kross Golden [Ale] 5,3% de Curacaví</li>
  <li>Kross Pilsner [Pilsner] 4,9% de Curacaví</li>
</ul>
```

Texto Plano

Hay mucha variedad en las cervezas locales de Chile. La sede de la marca "Austral" se encuentra en Punta Arenas. Austral fabrica una amplia gama de cervezas, incluyendo Larger (4,6%), Yagan (un ale de 5%) y un Pale Ale (5%). La cerveza de marca "Kunstmann" también es popular, en particular su cerveza "Torobayo" (un ale de 5% elaborado en Valdiva). La marca Kross, basada en Curacaví, también tiene una gama de cervezas populares como, por ejemplo, Kross 5 (un ale fuerte de 7,2%) Kross Golden (un ale de 5,3%) y Kross Pilsner (4,9%).

No estructurados

Bases de datos relacionales

- Mucha estructura (un esquema fijo)
- Muchas garantías (ACID)
- Generalmente centralizadas (viven en un servidor)

Pero existen más tipos de bases de datos…

Las **NoSQL**

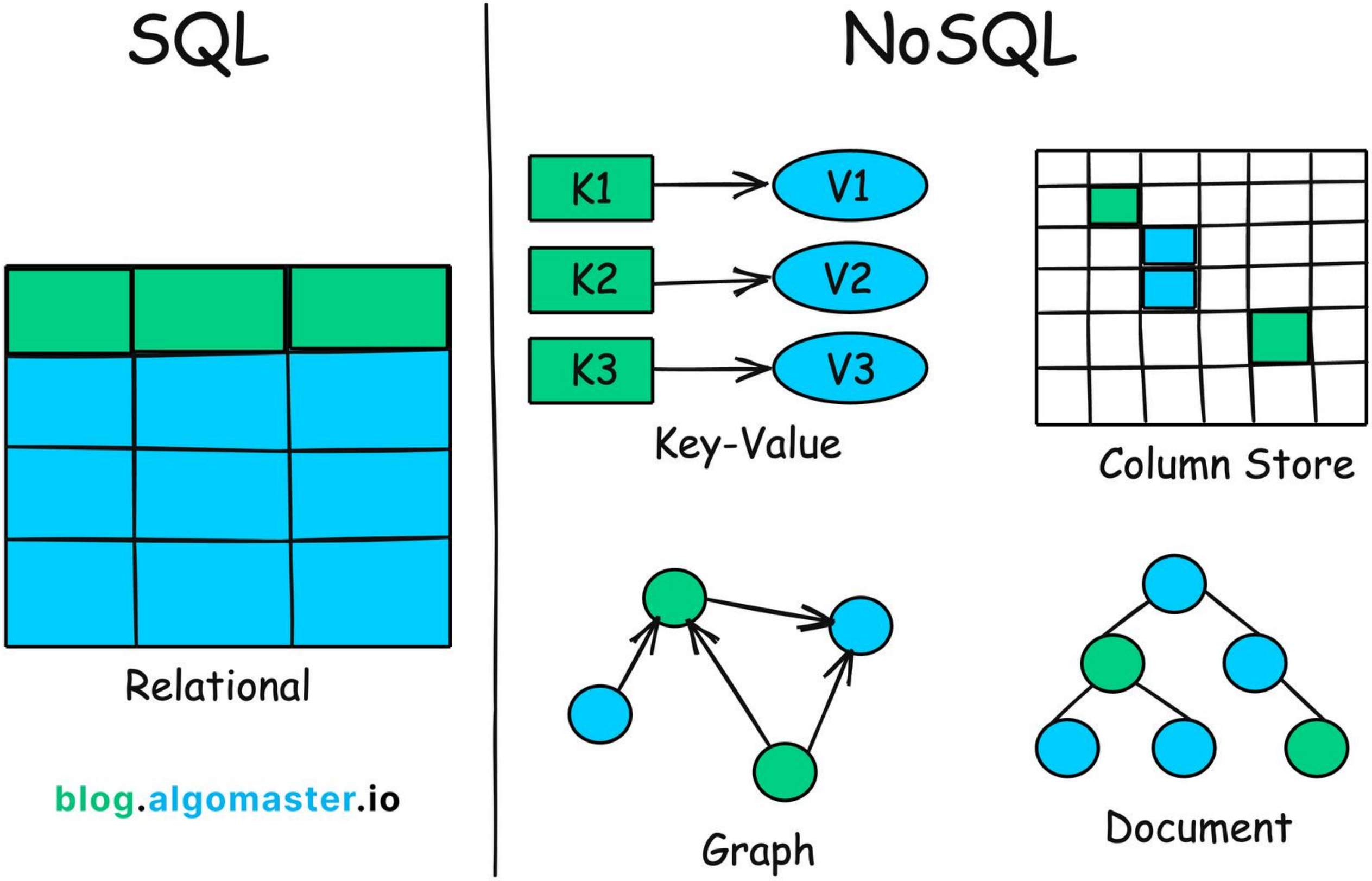
NoSQL

También llamadas Not Only SQL

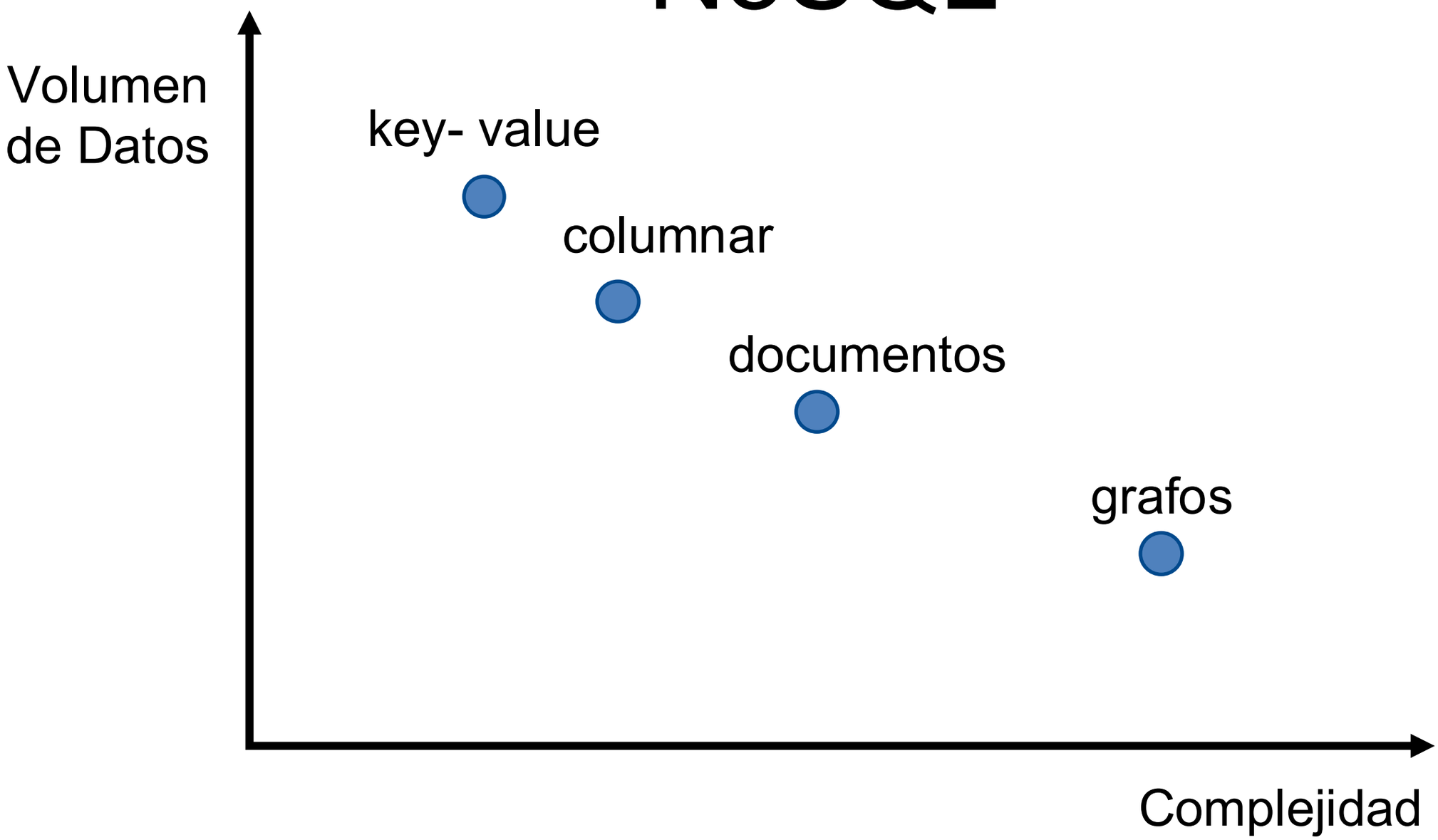
Término común para denominar bases de datos con nacidas desde 1998:

- Menos restricciones que el modelo relacional
- Menos esquema
- Menos garantías de consistencia
- Más adecuadas para la distribución

Aspecto	SQL (Relacional)	NoSQL (No Relacional)
Modelo de datos	Estructurado, basado en tablas con esquemas y relaciones fijas	Modelos de datos flexibles (documentos, clave-valor, columna, grafo)
Esquema	Rígido; debe definirse previamente con estructura estricta	Sin esquema o flexible; permite datos dinámicos y no estructurados
Escalabilidad	Escalado vertical (scale-up)	Escalado horizontal (scale-out) entre nodos distribuidos
Consistencia	Consistencia fuerte con transacciones ACID	Consistencia eventual, ajustable en algunos sistemas (ejemplo, MongoDB)
Lenguaje de consulta	SQL (estandarizado y potente para consultas complejas)	Varía (ejemplo: lenguaje de consulta de MongoDB, Cassandra Query Language)
Soporte de transacciones	ACID completo (Atomicidad, Consistencia, Aislamiento, Durabilidad)	Modelo BASE (Basicamente Disponible, Estado suave, Consistencia eventual); algunos soportan ACID en contextos limitados
Rendimiento	Optimizado para consultas complejas y relaciones; más lento en escrituras a gran escala	Optimizado para lecturas/escrituras de alto rendimiento, especialmente en sistemas distribuidos
Casos de uso	Mejor para datos estructurados y aplicaciones que requieren alta consistencia (ejemplo, sistemas financieros, ERP, CRM)	Mejor para datos no estructurados/semi-estructurados y sistemas distribuidos a gran escala (big data, analítica en tiempo real, redes sociales)
Desafíos de escalado	Difícil de fragmentar o particionar en varios servidores	Se pueden fragmentar y distribuir de forma sencilla
Integridad de datos	Alta; se aseguran relaciones e integridad mediante restricciones	Varía; la integridad suele gestionarse a nivel de aplicación
Uniones y relaciones	Permite realizar uniones complejas entre tablas y relaciones de clave externa	Las relaciones se gestionan de forma embebida (documentos) o de otra forma (base de datos de grafos)
Ejemplos	MySQL, PostgreSQL, Oracle, SQL Server	MongoDB, Cassandra, DynamoDB, Redis, Neo4j


SQL
NoSQL
K1
V1
K2
V2
K3
V3
Key-Value
Column Store
Relational
blog.algomaster.io
Graph
Document

Sabores de NoSQL



426 systems in ranking, November 2025

Rank Nov 2025	Rank Oct 2025	Rank Nov 2024	DBMS	Database Model	Score Nov 2025	Score Oct 2025	Score Nov 2024
1.	1.	1.	Oracle	Relational, Multi-model	1239.78	+27.01	-77.23
2.	2.	2.	MySQL	Relational, Multi-model	865.82	-13.84	-151.98
3.	3.	3.	Microsoft SQL Server	Relational, Multi-model	718.87	+3.82	-80.94
4.	4.	4.	PostgreSQL	Relational, Multi-model	651.36	+8.16	-2.97
5.	5.	5.	MongoDB	Multi-model	371.68	+3.66	-29.25
6.	6.	↑ 7.	Snowflake	Relational	197.84	-0.81	+55.35
7.	7.	↓ 6.	Redis	Key-value, Multi-model	145.09	+2.76	-3.55
8.	8.	↑ 14.	Databricks	Multi-model	131.50	+2.70	+45.04
9.	9.	9.	IBM Db2	Relational, Multi-model	119.28	-3.10	-2.47
10.	10.	↓ 8.	Elasticsearch	Multi-model	113.97	-2.69	-17.67
11.	↑ 12.	↓ 10.	SQLite	Relational	104.19	-0.36	+4.71
12.	↓ 11.	↓ 11.	Apache Cassandra	Wide column, Multi-model	102.71	-2.45	+5.00
13.	13.	↑ 15.	MariaDB	Relational, Multi-model	87.36	-0.41	+4.66
14.	14.	↓ 12.	Microsoft Access	Relational	78.29	-2.50	-13.03
15.	↑ 16.	↑ 16.	Microsoft Azure SQL Database	Relational, Multi-model	76.38	+0.90	-0.15
16.	↑ 18.	↓ 13.	Splunk	Search engine	76.10	+2.22	-12.37
17.	↓ 15.	17.	Amazon DynamoDB	Multi-model	75.78	-0.13	+3.38
18.	↓ 17.	18.	Apache Hive	Relational	72.87	-2.35	+21.37
19.	19.	19.	Google BigQuery	Relational	62.79	-0.42	+12.20
20.	20.	↑ 21.	Neo4j	Graph	52.36	-0.15	+9.66
21.	21.	↑ 24.	Apache Solr	Search engine, Multi-model	34.95	-0.36	+2.53
22.	22.	↑ 23.	Teradata	Relational, Multi-model	33.33	+0.21	-5.35
23.	23.	↓ 20.	FileMaker	Relational	32.79	-0.04	-9.99
24.	24.	↓ 22.	SAP HANA	Relational, Multi-model	32.22	+0.58	-6.88
25.	25.	25.	SAP Adaptive Server	Relational, Multi-model	27.08	-0.08	-4.41
26.	26.	↑ 34.	Apache Spark (SQL)	Relational	23.09	-0.68	+6.31
27.	27.	27.	Microsoft Azure Cosmos DB	Multi-model	22.55	-0.16	-1.40
28.	28.	28.	InfluxDB	Time Series, Multi-model	22.09	+0.18	+0.61
29.	29.	29.	PostGIS	Spatial, Multi-model	21.48	+0.44	+1.86
30.	30.	↑ 35.	ClickHouse	Relational, Multi-model	21.13	+0.95	+4.63

https://db-engines.com/en/ranking

BD Key - Value

Independientemente del esquema

- Arquitectura almacena información por medio de pares
- Cada par tiene una llave (identificador) y un valor

BD Key - Value

Operaciones cruciales:

- put(key,value)
- get(key)
- delete(key)

Key	Value
Chile	Santiago
Inglaterra	Londres
Escocia	Edinburgo
Francia	Paris
Alemania	Berlin
…	…

BD Key - Value

- Son grandes tablas de hash persistentes
- Esta categoría es difusa, pues muchas de las aplicaciones de otros tipos de BD usan key - value y hashing hasta cierto punto

Ejemplo más importante: Amazon Dynamo, otro es Redis

BD Key - Value

Puede representar cualquier valor

cartID	value
11789	usrID: "Juan", ítem: "Magic the Gathering Deck", value: …
12309	usrID: "Domagoj", item: "APEX XTX50 regulator set", value: …
…	…

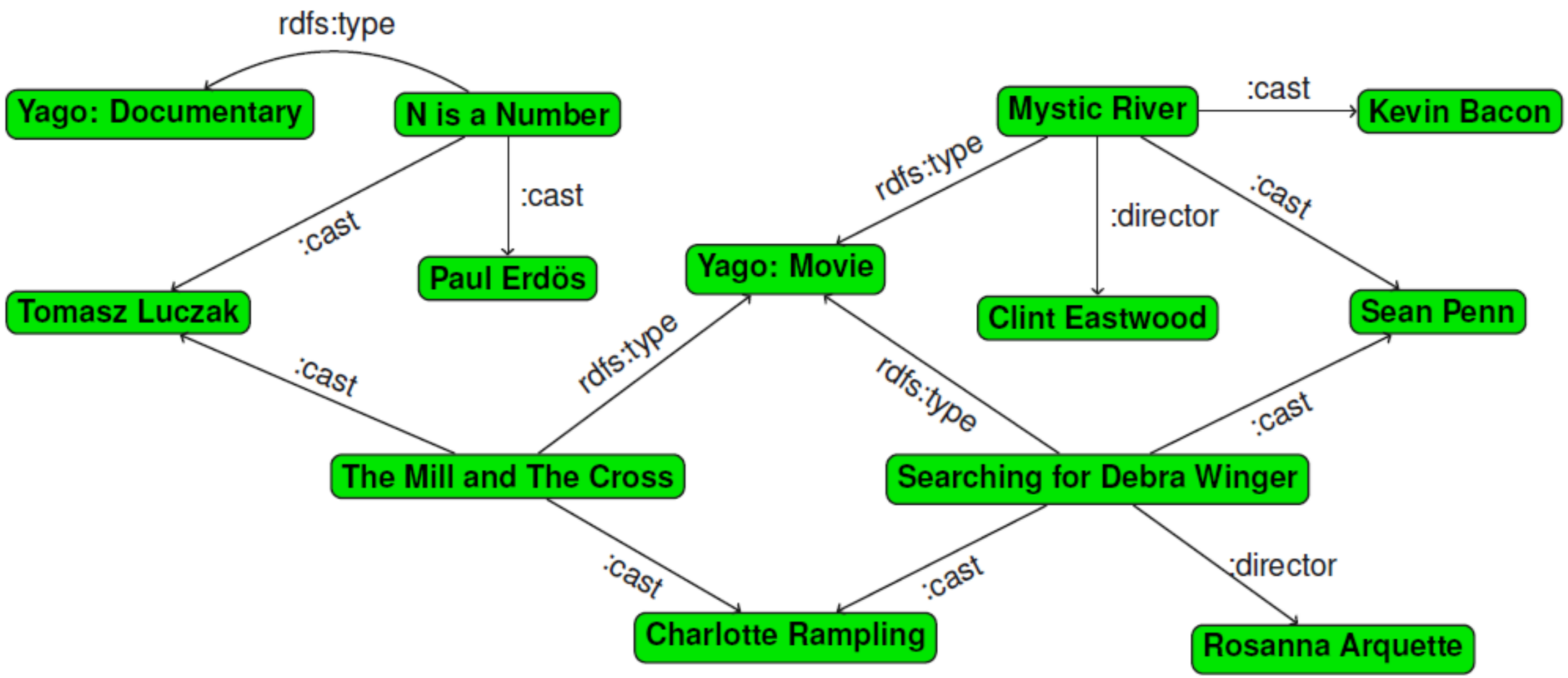
BD de Grafos y RDF

(Resource Description Framework)

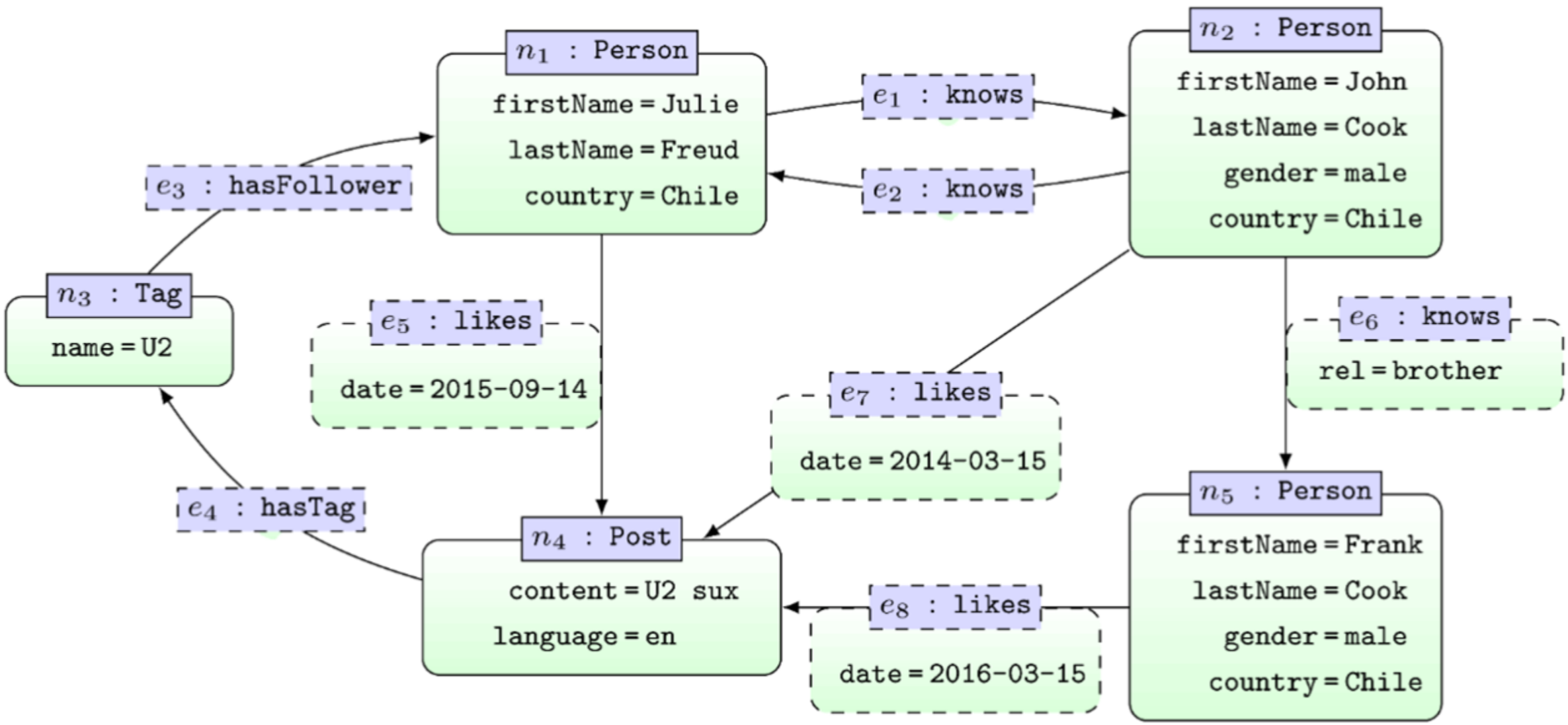
Especializadas para guardar relaciones

- En general, almacenan sus datos como property graphs
- Algunos ejemplos son Neo4J, Virtuoso, Jena, Blazegraph

BD de Grafos y RDF



BD de Grafos y RDF



BD Orientadas a Documentos

Especializadas en documentos

- CouchDB, **MongoDB** (estas y otras BD almacenan sus datos en documentos JSON)
- JSON no es el único estándar de documentos (por ejemplo, existe también XML)

Ahora nos centraremos en **BD Orientadas a documentos**, específicamente en **MongoDB**

BD Orientadas a Documentos

Parecidas a key-value stores:

- El valor es ahora un documento (JSON)
- Pueden agrupar documentos (colecciones)
- Lenguaje de consultas mucho más poderoso

BD Orientadas a Documentos

Usuarios	
Key	JSON

1	{
	"uid": 1,
	"name": "Adrian",
	"last_name": "Soto",
	"ocupation": "Delantero de Cobreloa",
	"follows": [2,3],
	"age": 24
	}

2	…
…	…

JSON

Su nombre viene de JavaScript Object Notation

Estándar de intercambio de datos semiestructurados / datos en la Web

- JSON se acopla muy bien a los lenguajes de programación

JSON - Ejemplo


```
{
  "statuses": [
    {
      "id": 725459373623906304,
      "text": "@visitlondon: Have you been to any of these
              quirky London museums? https://t.co/tnrar8UttZ",
      "retweeted_status": {
        "metadata": {
          "result_type": "recent",
          "iso_language_code": "en"
        },
        "retweet_count": 239,
        "retweeted": false
      }
    }
  ]
}
```

JSON

La base son los pares key - value

```
{
  “nombre": “Matías”, “apellido": “Jünemann”
}
```

Valores pueden ser:

- Números
- Strings (entre comillas)
- Valores booleanos
- Arreglos (por definir)
- Objetos (por definir)
- null

JSON - Sintaxis

Los objetos se escriben entre {} y contienen una cantidad arbitraria de pares key - value

```
{
  “nombre": “Matías”, “apellido": “Jünemann”
}
```

JSON - Sintaxis

Los arreglos se escriben entre [] y contienen valores

```
{
  “profesores”: [
   {“nombre”: “Juan”, “apellido": “Reutter”},
   {“nombre”: “Cristian”, “apellido": “Riveros”},
   {“nombre”: “Marcelo”, “apellido": “Arenas”}
  ]
}
```

JSON vs SQL

SQL:

- Esquema de datos
- Lenguajes de consulta independientes del código

JSON:

- Más flexible, no hay que respetar necesariamente un esquema
- Más tipos de datos (como arreglos)
- Human - Readable

Ejemplo

Almacenar varios roles a personas

ID	Persona	rol
1	Juan Soto	Paciente y Medico
2	Pedro Perez	Paciente

SQL →

ID	Persona
1	Juan Soto
2	Pedro Perez

1 — N

#	ID	rol
1	1	Paciente
2	1	Medico
3	2	Paciente

mongoDB

colección

documento

ID: 1
Nombre:”Juan Soto”
Rol:[“Paciente”, “Médico”]

documento

ID: 2
Nombre:”Pedro Perez”
Rol:”Paciente”

BD de documentos: ¿para qué?

Especializadas en documentos: almacenan muchos documentos JSON

- Si quiero libros: un documento JSON por libro
- Si quiero personas: un documento JSON por persona

Notar que esto es altamente jerárquico

BD de documentos

Qué hacen bien:

- Si quiero un libro o persona en particular
- Cruce de información **simple**

Muy útiles a la hora de desplegar información en la web

Además, verse como un **cache** de una pueden BD relacional

Caché de BD SQL

Students

StudentID	Nombre	Carrera
1	Alice Cooper	Computación
2	David Bowie	Todas
3	Charly García	Ingeniería Civil
…	…	…

Courses

courseID	name	year
IIC2413	Databases	2020
IMT3830	Game Theory	2020
…	…	…

Takes

courseID	StudentID
IIC2413	1
IIC2413	2
IMT3830	2
…	…

Caché de BD SQL

Lista de alumnos por curso:

- SQL tiene que hacer un join
- En BD documentos prepararemos esta información

Caché de BD SQL

Colección "**Courses**"

```
{
    "courseID": IIC2413,
    "name": "Databases",
    "year": 2020,
    "students":[
        {
            "studentID": 1,
            "name": "Alice Cooper"
        },
        {
            "studentID": 2,
            "name": "David Bowie"
        },
        ...
    ]
}
```

```
{
    "courseID": IMT3830,
    "name": "Game Theory",
    "year": 2020,
    "students":[
        {
            "studentID": 2,
            "name": "David Bowie"
        },
        {
            "studentID": 3,
            "name": "Charly García"
        },
        ...
    ]
}
```

BD de documentos

Qué hacen bien en este caso:

- Si quiero lista de alumnos de un curso
- Si quiero nombres de todos los cursos
- Si quiero todo los cursos tomados por “David”

BD de documentos

Qué hacen mal:

- Manejo de información que cambia mucho
- Cruce de información no trivial

BD de documentos

Colección "**Courses**"

- Todos los alumnos que toman IIC2413 y IMT3830

```
{
    "courseID": IIC2413,
    "name": "Databases",
    "year": 2020,
    "students":[
        {
            "studentID": 1,
            "name": "Alice Cooper"
        },
        {
            "studentID": 2,
            "name": "David Bowie"
        },
        ...
    ]
}
```

```
{
    "courseID": IMT3830,
    "name": "Game Theory",
    "year": 2020,
    "students":[
        {
            "studentID": 2,
            "name": "David Bowie"
        },
        {
            "studentID": 3,
            "name": "Charly García"
        },
        ...
    ]
}
```

BD de documentos

Colección "**Courses**"

- Todos los alumnos que toman IIC2413 y IMT3830

Efectivamente hay que hacer un nested loop join:

- Iterar por todos los alumnos de IMT3830
- Iterar por todos los alumnos de IIC2413
- Ver si hacen el join

MongoDB soporta JavaScript y Python

- Se puede hacer, pero no es elegante

En resumen

BD de documentos:

- Útiles para despliegue de información estática
- Búsquedas simples
- Cruces muy sencillos

BD SQL:

- Información cambia mucho
- Tengo que hacer cruces cada rato
- Necesito ACID

BD Documentos y BASE

- Distintas aplicaciones en una misma base de datos acceden a distintos documentos al mismo tiempo
- En general diseñadas para montar varias instancias que (en teoría) tienen la misma información
- Propagan updates en forma descoordinada

Proveen **Consistencia Eventual**: Con el tiempo, todos los nodos del sistema llegarán a un estado consistente, permitiendo alta disponibilidad y tolerancia a fallos, a costa de no tener consistencia inmediata.

Consistencia Eventual

Pero la consistencia eventual puede generar problemas:

Si dos aplicaciones intentan acceder al mismo documento en MongoDB, estas pueden ser versiones diferentes del documento

NoSQL	KeyValue	Grafos	Documentos	MongoDB	Text search&retreival

MongoDB

MongoDB

Es una base de datos NoSQL orientada a documentos que almacena datos en formato JSON (BSON). Estas son sus características:

- **Flexibilidad de Esquema**: Permite almacenar documentos con diferentes estructuras sin migraciones complejas.
- **Escalabilidad Horizontal**: Distribuye datos en múltiples servidores para mejorar rendimiento y capacidad.
- **Alto Rendimiento**: Optimizado para operaciones rápidas de lectura y escritura.
- **Replicación y Alta Disponibilidad**: Asegura disponibilidad y recuperación ante fallos mediante réplicas.

Usuarios	
Key	JSON

60bfd90e002ce228636e506b	{
	"_id": ObjectId('60bfd90e002ce228636e506b'),
	"uid": 1,
	"name": "Adrian",
	"last_name": "Soto",
	"ocupation": "Delantero de Cobreloa",
	"follows": [2,3],
	"age": 24
	}

60bfd90e002ce228636e5215	...
...	...

Colección: una agrupación de documentos similares

Usuarios	
Key	JSON

60bfd90e002ce228636e506b	{
	"_id": ObjectId('60bfd90e002ce228636e506b'),
	"uid": 1,
	"name": "Adrian",
	"last_name": "Soto",
	"ocupation": "Delantero de Cobreloa",
	"follows": [2,3],
	"age": 24
	}

60bfd90e002ce228636e5215	...
...	...

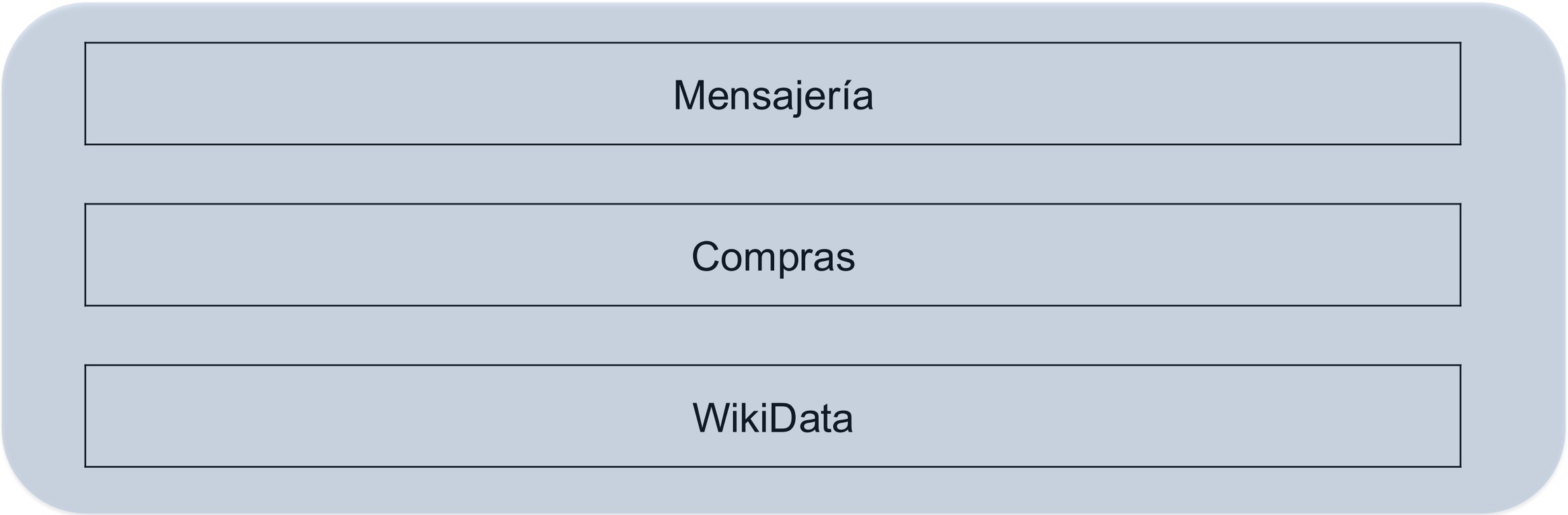
Base de Datos: contienen colecciones relacionadas

Usuarios	
Key	JSON
…	…

Mensajes	
Key	JSON
…	…

Likes	
Key	JSON
…	…

NoSQL	KeyValue	Grafos	Documentos	MongoDB	Text search&retreival



Un servidor contiene varias bases de datos

Consultando a MongoDB

show dbs … muestra bases de datos disponibles
use dbName … ahora usamos base de datos dbName
show collections … colecciones en nuestra base de datos

db.colName.find() … todos los documentos en la colección colName

db.colName.find().pretty() … muentra los datos de mejor forma

db.colName.find({"name": "Adrian"}) … selección

db.colName.find({"age": {$gte:23}}) … selección

db.colName.find({"age": {$gte:23}},{"name":1}) … proyección

NoSQL	KeyValue	Grafos	Documentos	MongoDB	Text search&retreival

Full-text search & retrieval

¿Cómo buscar texto?

Cuando buscamos “Chilean Mammal”:

- El sistema encuentra todos los documentos que tienen Chilean y Mammal

¿Pero cómo lo hacemos para ordenar los resultados?
Puede ser problemático en grandes bases de datos

Índices Invertidos

Para hacer la búsqueda eficiente utilizamos **índices invertidos**

Para cada palabra del universo de documentos, guardamos punteros que nos indican dónde están los documentos

Índices Invertidos

	Documento 1	Documento 2	Documento 3	...
Chile	1	0	1	
of	1	1	1	
town	0	0	1	
commune	0	1	0	
...				

Text Search en MongoDB

Un índice especial … permite búsqueda rápida de texto

db.colName.createIndex({"attributeName":"text"})

db.users.find({$text: {$search: "Delantero de Cobreloa"}})

Ver más en: https://www.youtube.com/watch?v=vR97-4UG7x0

TF – IDF

(Frecuencia del Término - Frecuencia Inversa de los Documentos)

mide la importancia de una palabra en un documento en comparación con un conjunto de documentos.

TF – IDF

Principio 1:

- El puntaje es proporcional a la cantidad de veces que aparece la palabra en el documento

Principio 2:

- El puntaje es inversamente proporcional a la cantidad de documentos en los que aparece la palabra

TF – IDF

Term Frequency:

- $F_D(t)$ = Número de veces que aparece t en D

Inverse Document Frequency:

- IDF(t) = log(número de documentos / númerode documentos en los que aparece t)

Entonces:

$$\textbf{TF-IDF} = F_D(t) \cdot IDF(t)$$

TF - IDF

Ejemplo

D1: Ojo por ojo, diente por diente

D2: Ojo por ojo, y el mundo acabará ciego

D3: Si luchas contra el mundo, ponte del lado del mundo

Entonces, calculamos el TF-IDF de “ojo” y “mundo” para cada documento…

TF – IDF: Ejemplo

Paso 1, Calcular TF: El TF se calcula como el número de veces que aparece un término en un documento dividido por el número total de términos en el documento.

Documento D1: "Ojo por ojo, diente por diente"

- Total de palabras: 6
- TF("ojo") = 2/6 = 0.333
- TF("mundo") = 0/6 = 0

Documento D2: "Ojo por ojo, y el mundo acabará ciego"

- Total de palabras: 8
- TF("ojo") = 2/8 = 0.25
- TF("mundo") = 1/8 = 0.125

Documento D3: "Si luchas contra el mundo, ponte del lado del mundo"

- Total de palabras: 10
- TF("ojo") = 0/10 = 0
- TF("mundo") = 2/10 = 0.2

TF – IDF: Ejemplo

Paso 2, calcular IDF: El IDF se calcula como el logaritmo del número total de documentos dividido por el número de documentos que contienen el término.

- Total de documentos: 3
- Documentos que contienen "ojo": 2 (D1 y D2)
- Documentos que contienen "mundo": 2 (D2 y D3)
- IDF("ojo") = log(3/2) = 0.176
- IDF("mundo") = log(3/2) = 0.176

TF – IDF: Ejemplo

Paso 3, Calcular TF-IDF: El TF-IDF se calcula multiplicando el TF por el IDF.

TF-IDF para “ojo”:

- Documento D1: 0.333 * 0.176 = 0.059
- Documento D2: 0.25 * 0.176 = 0.044
- Documento D3: 0 * 0.176 = 0

TF-IDF para “mundo”:

- Documento D1: 0 * 0.176 = 0
- Documento D2: 0.125 * 0.176 = 0.022
- Documento D3: 0.2 * 0.176 = 0.035

TF – IDF

Hay distintas funciones para TF e IDF

Generalmente se incorporan funciones para Stemming y Stop Words para procesamiento de lenguaje natural

Cada compañía tiene su receta, depende además del idioma

TF - IDF

Búsqueda de documentos

Se genera una matriz en donde las dimensiones son las palabras y los documentos

Cada “casillero” señala el TF-IDF de la palabra en cada documento

Cuando un usuario busca una frase, se genera un vector y se retornan los documentos con vectores más similares

NoSQL	KeyValue	Grafos	Documentos	MongoDB	Text search&retreival

Map Reduce

Map Reduce

- Algoritmo eficiente para computación paralela/distribuida
- Paradigma de programación - mezcla de datos con controlador
- Sacrificios para acelerar computación

Computación Distribuida

- Datos tan grandes que no caben en un computador
- Repartidos en muchos servidores
- Cada servidor no conoce lo que tiene el resto
- No podemos dar el lujo de comunicar todos los datos

Map Reduce

Ejemplo: ver cuantas veces ocurre cada palabra en un texto T

¿Cómo hacer esto en un archivo de 100 **petabytes***?

*100 petabytes = 100000000 gigabytes

Map Reduce

- **Map**: recibe datos y genera pares key – values
- **Reduce**: recibe pares con el mismo key y los agrega
- **Shuffle**: transfiere los datos desde mappers a reducers

Map Reduce - Arquitectura

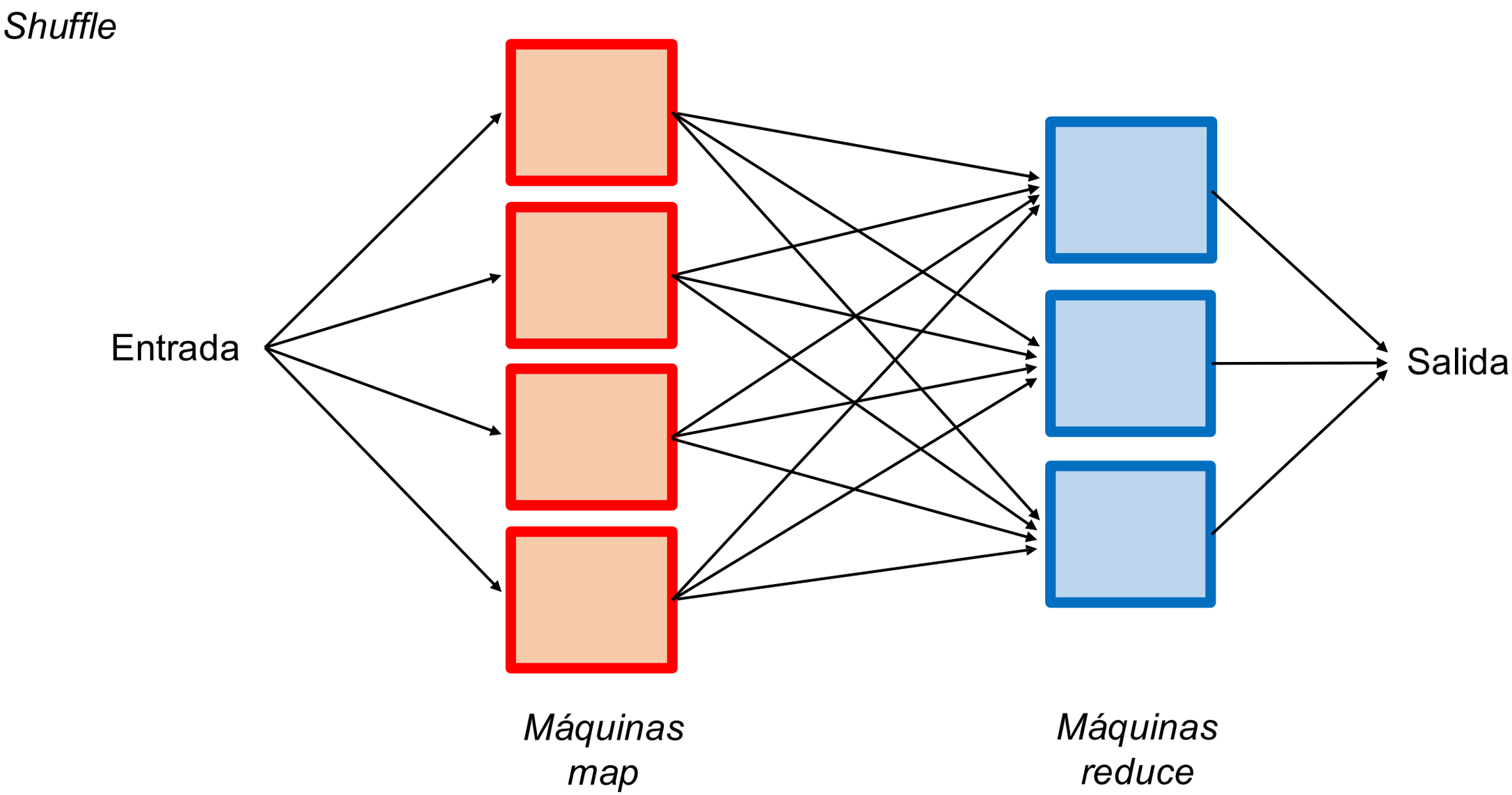
Mappers:

- Nodos encargados de hacer Map
- Reciben parte del documento y lo envían a los reducers (a través de **shuffle**)

Reducers:

- Nodos encargados de hacer Reduce
- Reciben los Map y los agregan
- El output es la unión de cada Reducer

Map Reduce



Map Reduce - Ejemplo

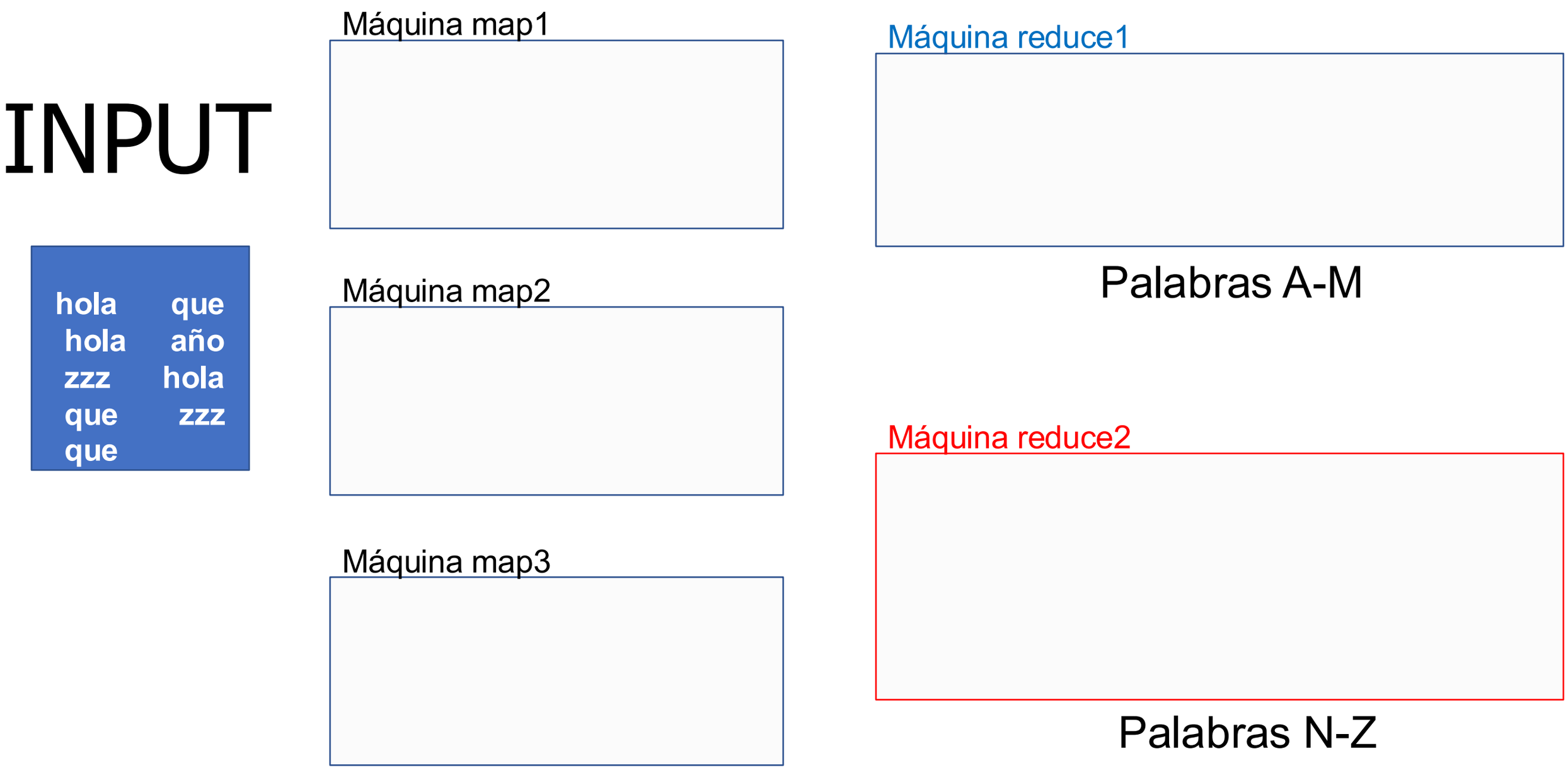
¿Cuántas veces cada palabra ocurre en un archivo de texto grande? Para saberlo, tenemos:

Map:

- Recibe un pedazo de texto
- Por cada palabra, emite el par (palabra, número de ocurrencias)

Reduce:

- Cada reduce recibe todos los pares asociados a la misma palabra
- Junta todos estos pares y suma las ocurrencias


INPUT
hola que
hola año
zzz hola
que zzz
que
Máquina map1
Máquina map2
Máquina map3
Máquina reduce1
Palabras A-M
Máquina reduce2
Palabras N-Z

SEPARAR INPUT

hola que hola
año zzz hola
que zzz que

Máquina map1

Máquina map2

Máquina map3

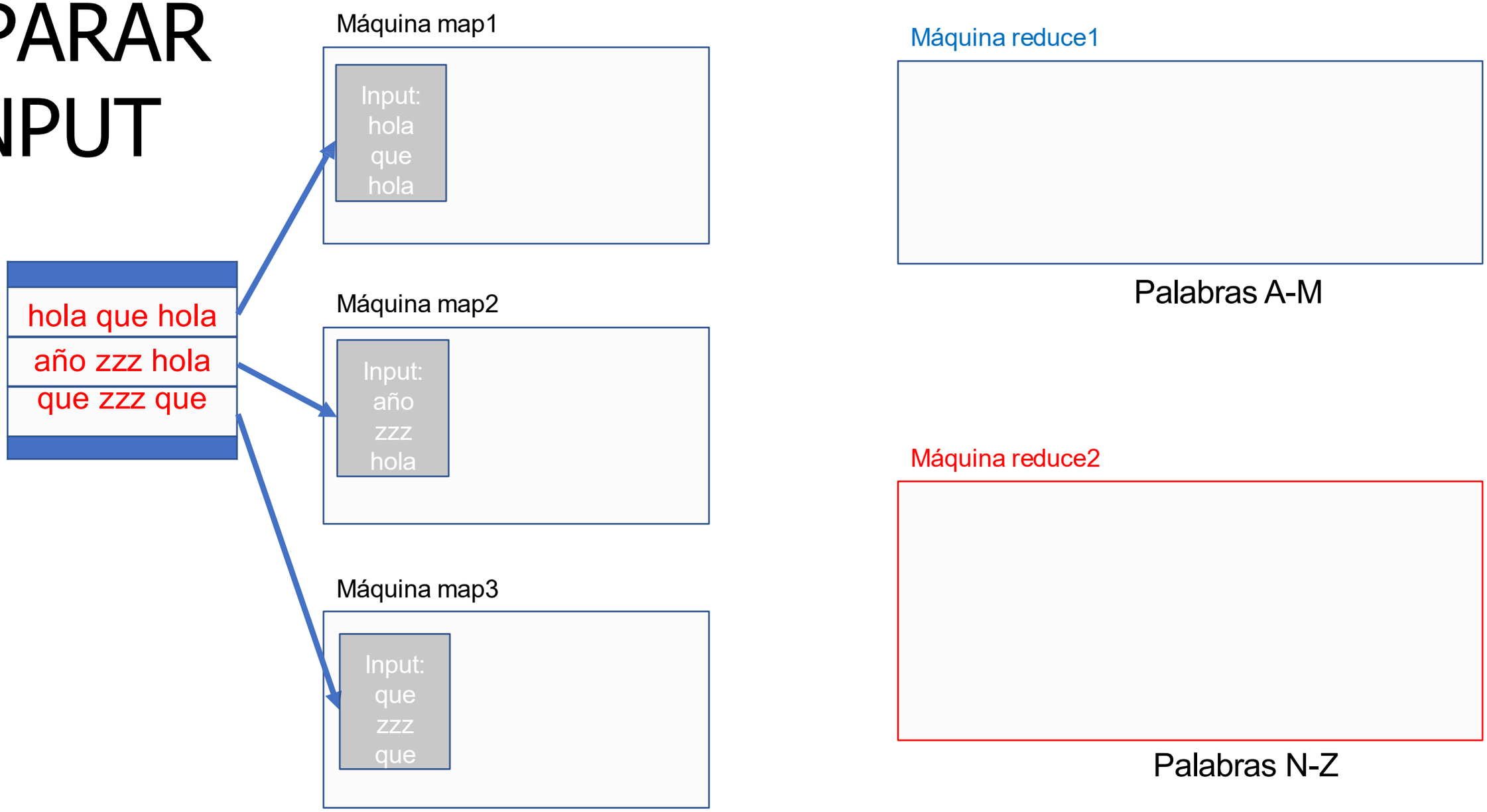
Máquina reduce1

Palabras A-M

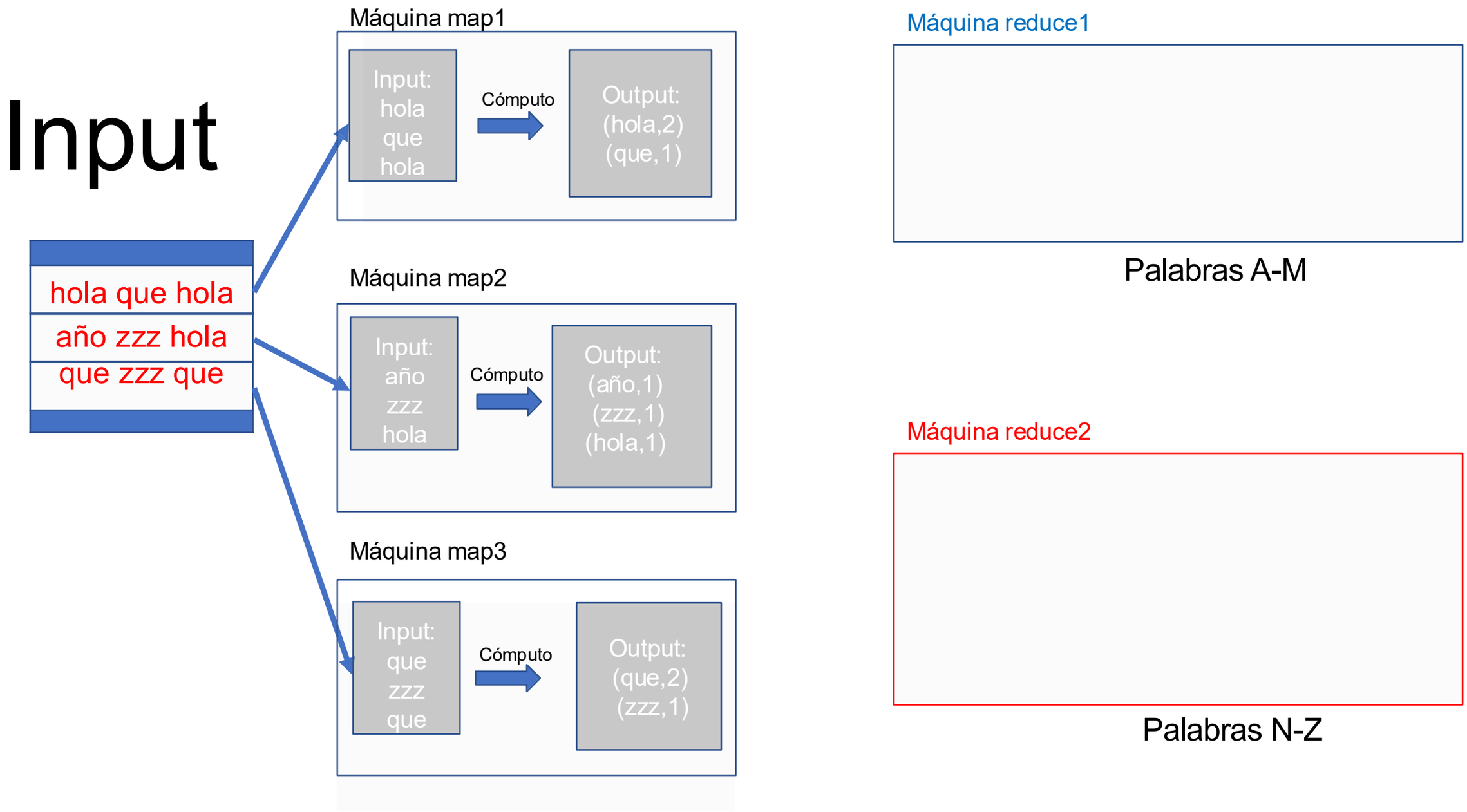
Máquina reduce2

Palabras N-Z

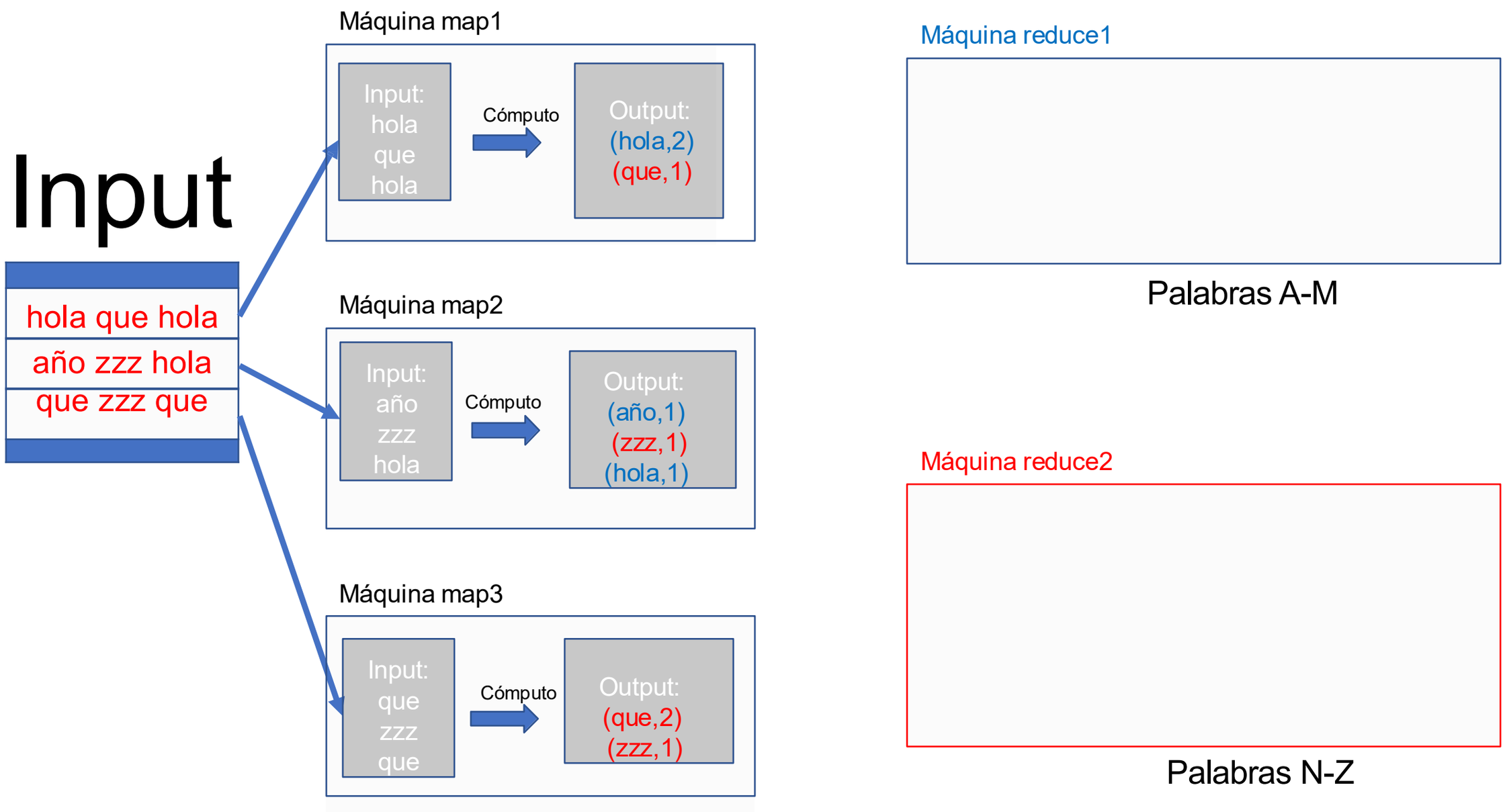
SEPARAR INPUT



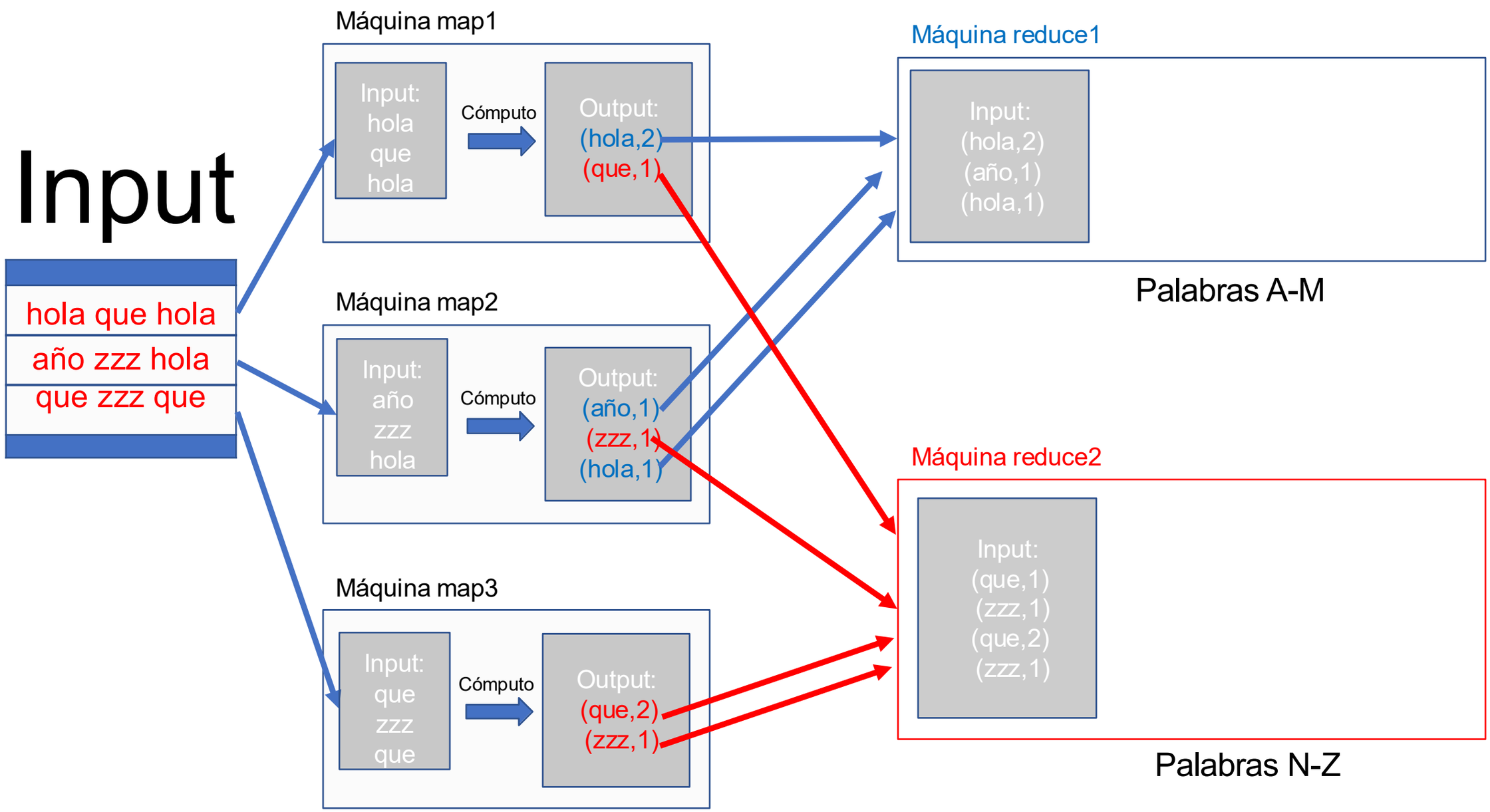
MAP


Input
hola que hola
año zzz hola
que zzz que
Máquina map1
Input: hola que hola
Cómputo
Output: (hola,2) (que,1)
Máquina map2
Input: año zzz hola
Cómputo
Output: (año,1) (zzz,1) (hola,1)
Máquina map3
Input: que zzz que
Cómputo
Output: (que,2) (zzz,1)
Máquina reduce1
Palabras A-M
Máquina reduce2
Palabras N-Z

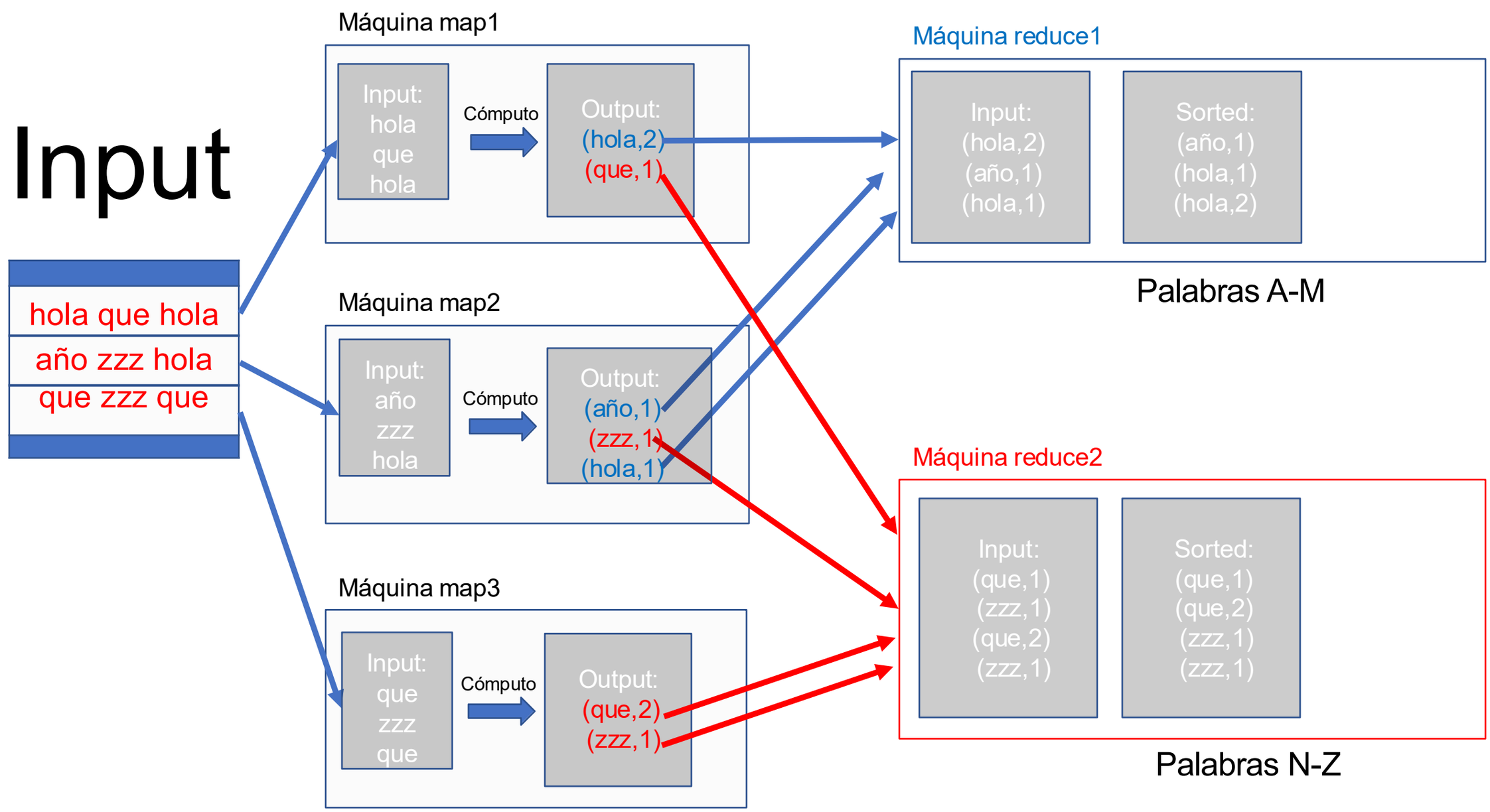
SHUFFLE


Input
hola que hola
año zzz hola
que zzz que
Máquina map1
Input: hola que hola
Cómputo
Output: (hola,2) (que,1)
Máquina map2
Input: año zzz hola
Cómputo
Output: (año,1) (zzz,1) (hola,1)
Máquina map3
Input: que zzz que
Cómputo
Output: (que,2) (zzz,1)
Máquina reduce1
Palabras A-M
Máquina reduce2
Palabras N-Z

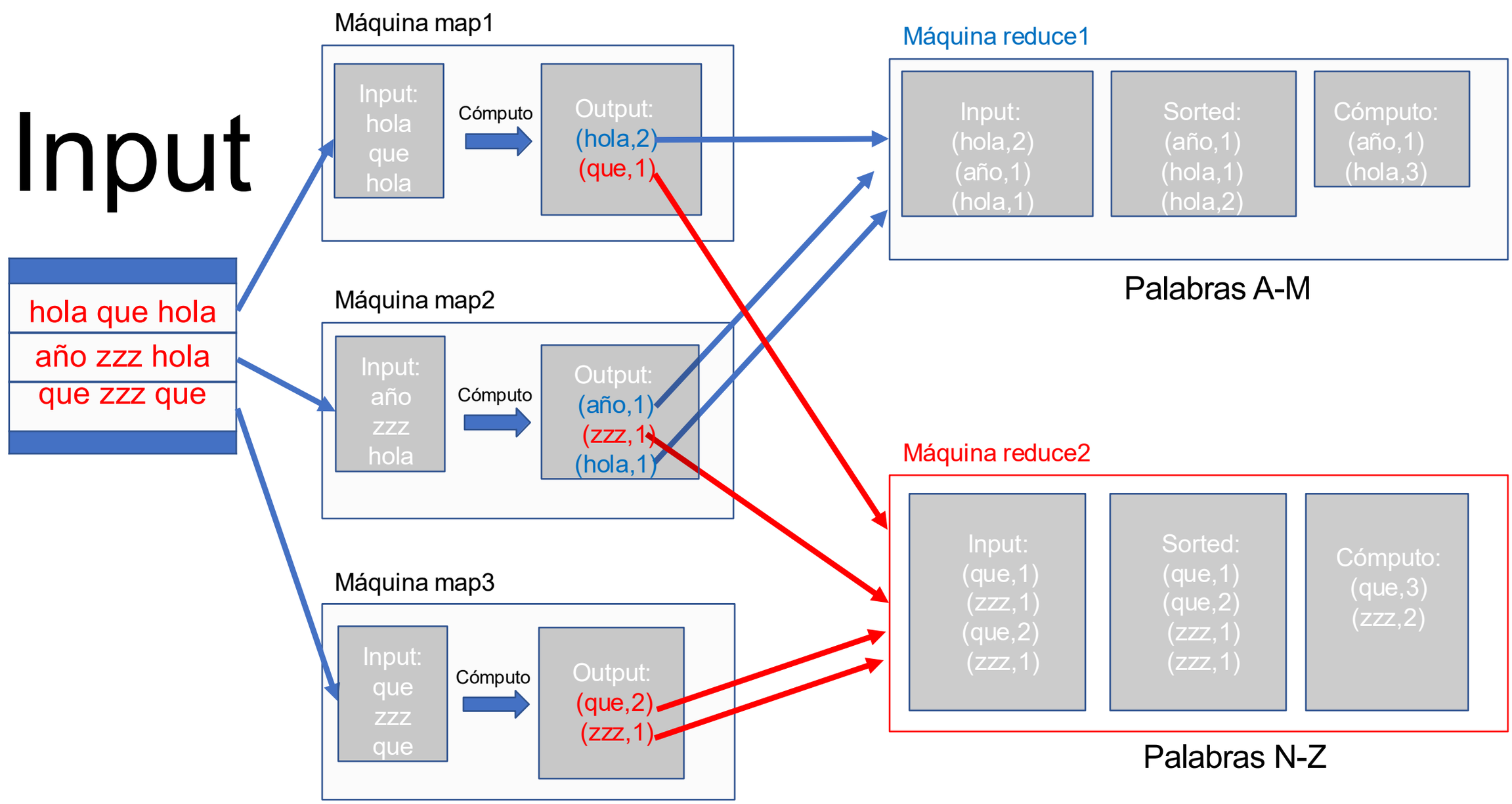
SHUFFLE

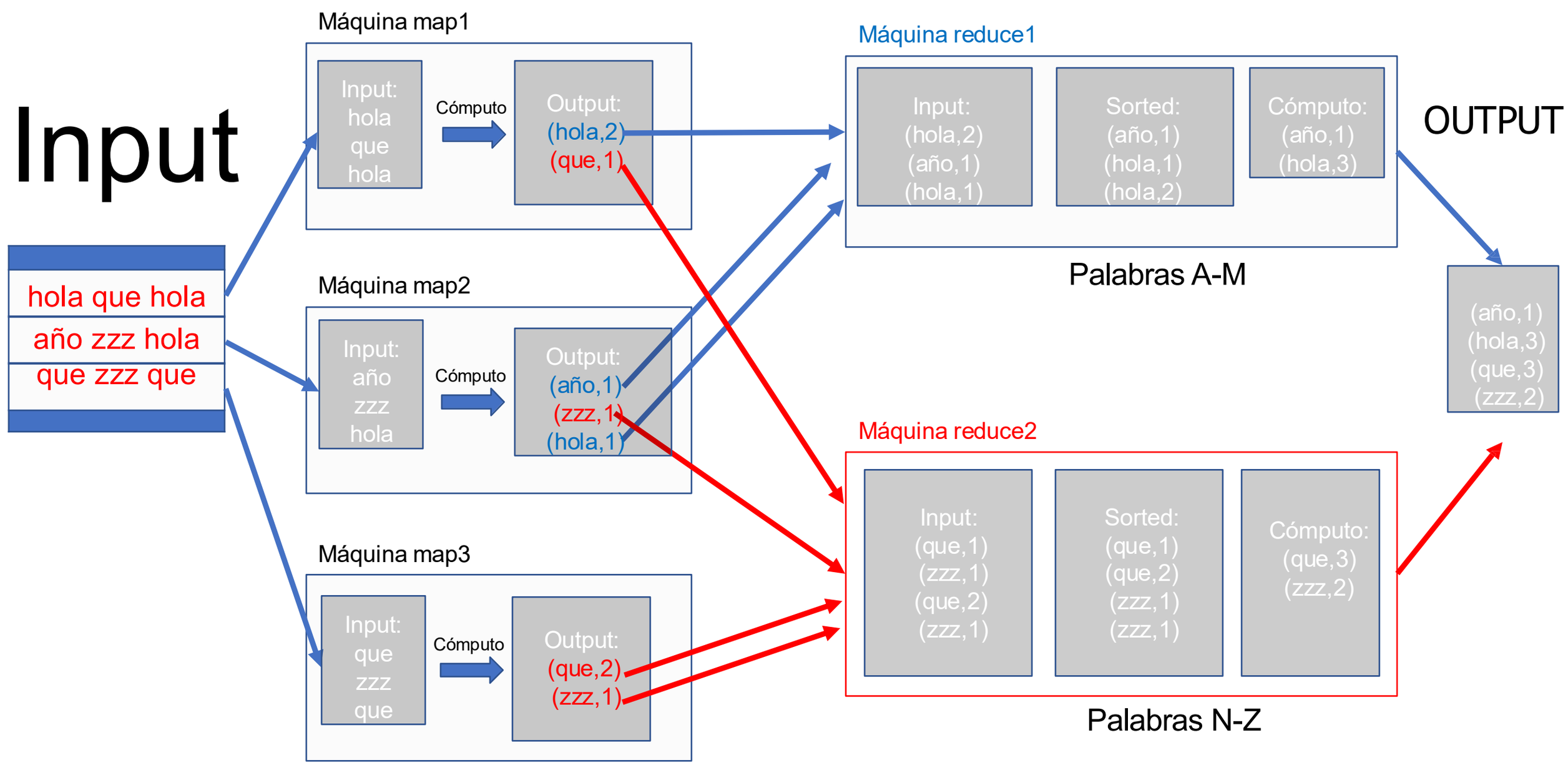

Input
hola que hola
año zzz hola
que zzz que
Máquina map1
Input: hola que hola
Cómputo
Output: (hola,2) (que,1)
Máquina map2
Input: año zzz hola
Cómputo
Output: (año,1) (zzz,1) (hola,1)
Máquina map3
Input: que zzz que
Cómputo
Output: (que,2) (zzz,1)
Máquina reduce1
Input: (hola,2) (año,1) (hola,1)
Palabras A-M
Máquina reduce2
Input: (que,1) (zzz,1) (que,2) (zzz,1)
Palabras N-Z

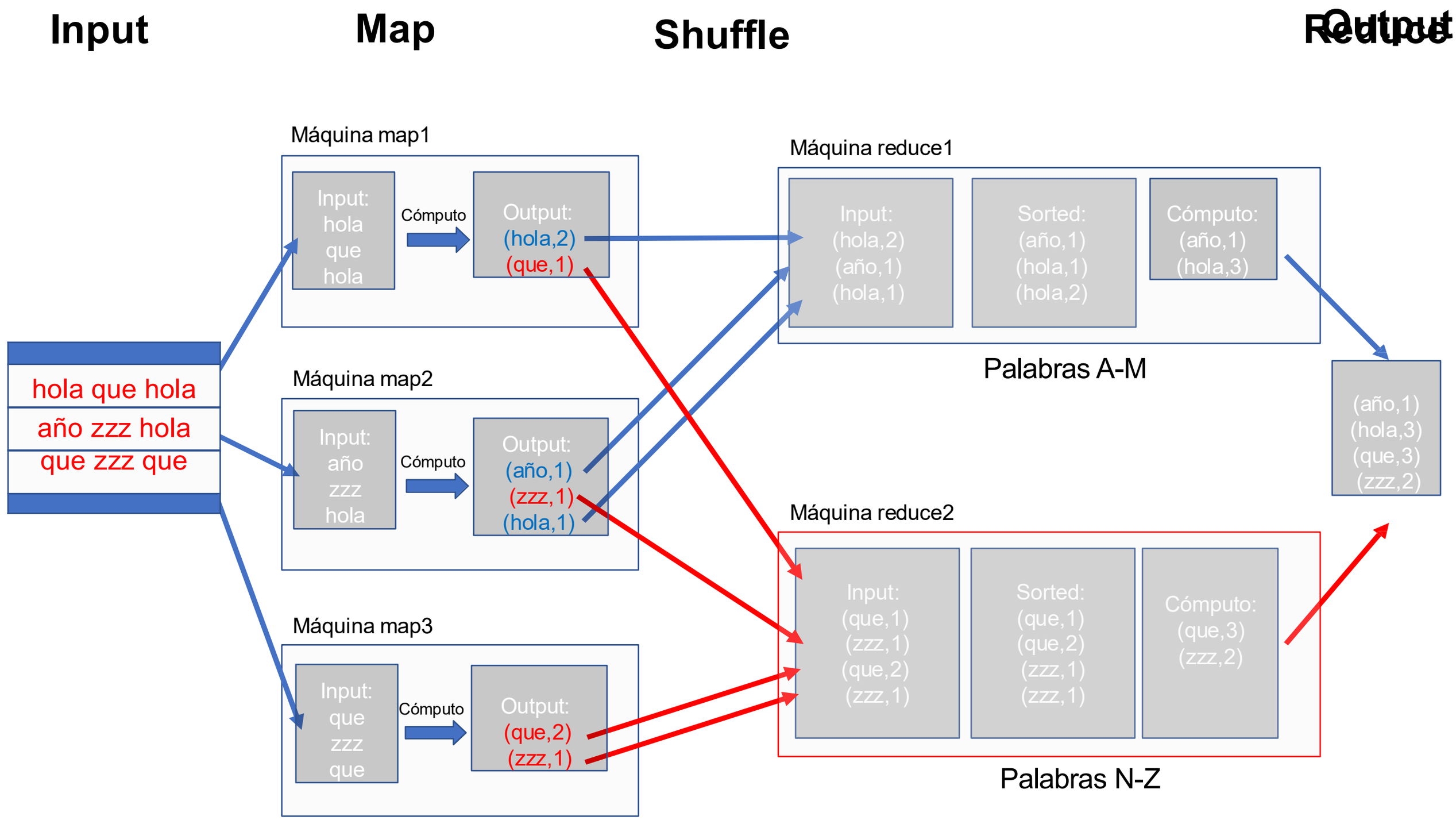
REDUCE


Input
hola que hola
año zzz hola
que zzz que
Máquina map1
Input: hola que hola
Cómputo
Output: (hola,2) (que,1)
Máquina map2
Input: año zzz hola
Cómputo
Output: (año,1) (zzz,1) (hola,1)
Máquina map3
Input: que zzz que
Cómputo
Output: (que,2) (zzz,1)
Máquina reduce1
Input: (hola,2) (año,1) (hola,1)
Sorted: (año,1) (hola,1) (hola,2)
Palabras A-M
Máquina reduce2
Input: (que,1) (zzz,1) (que,2) (zzz,1)
Sorted: (que,1) (que,2) (zzz,1) (zzz,1)
Palabras N-Z

REDUCE


Input
hola que hola
año zzz hola
que zzz que
Máquina map1
Input: hola que hola
Cómputo
Output: (hola,2) (que,1)
Máquina map2
Input: año zzz hola
Cómputo
Output: (año,1) (zzz,1) (hola,1)
Máquina map3
Input: que zzz que
Cómputo
Output: (que,2) (zzz,1)
Máquina reduce1
Input: (hola,2) (año,1) (hola,1)
Sorted: (año,1) (hola,1) (hola,2)
Cómputo: (año,1) (hola,3)
Palabras A-M
Máquina reduce2
Input: (que,1) (zzz,1) (que,2) (zzz,1)
Sorted: (que,1) (que,2) (zzz,1) (zzz,1)
Cómputo: (que,3) (zzz,2)
Palabras N-Z


Input
hola que hola
año zzz hola
que zzz que
Máquina map1
Input: hola que hola
Cómputo
Output: (hola,2) (que,1)
Máquina map2
Input: año zzz hola
Cómputo
Output: (año,1) (zzz,1) (hola,1)
Máquina map3
Input: que zzz que
Cómputo
Output: (que,2) (zzz,1)
Máquina reduce1
Input: (hola,2) (año,1) (hola,1)
Sorted: (año,1) (hola,1) (hola,2)
Cómputo: (año,1) (hola,3)
Palabras A-M
Máquina reduce2
Input: (que,1) (zzz,1) (que,2) (zzz,1)
Sorted: (que,1) (que,2) (zzz,1) (zzz,1)
Cómputo: (que,3) (zzz,2)
Palabras N-Z
OUTPUT
(año,1) (hola,3) (que,3) (zzz,2)


Input
Map
Shuffle
Reduce
Output
Máquina map1
Input: hola que hola
Cómputo
Output: (hola,2) (que,1)
Máquina map2
Input: año zzz hola
Cómputo
Output: (año,1) (zzz,1) (hola,1)
Máquina map3
Input: que zzz que
Cómputo
Output: (que,2) (zzz,1)
hola que hola
año zzz hola
que zzz que
Máquina reduce1
Input: (hola,2) (año,1) (hola,1)
Sorted: (año,1) (hola,1) (hola,2)
Cómputo: (año,1) (hola,3)
Palabras A-M
Máquina reduce2
Input: (que,1) (zzz,1) (que,2) (zzz,1)
Sorted: (que,1) (que,2) (zzz,1) (zzz,1)
Cómputo: (que,3) (zzz,2)
Palabras N-Z
(año,1) (hola,3) (que,3) (zzz,2)

Map Reduce - Ejemplo: Join

¿Cómo hago un join con Map Reduce?

- Modelo: un archivo con el nombre de la tabla y sus tuplas
- Map: Agrupo por el atributo que hace el join
- Reduce: Hago el producto cruz para las tablas distintas

INPUT

R

A	B
1	1
1	3
3	2
3	3

S

B	C
2	4
2	7
3	8
3	9

Máquina map1

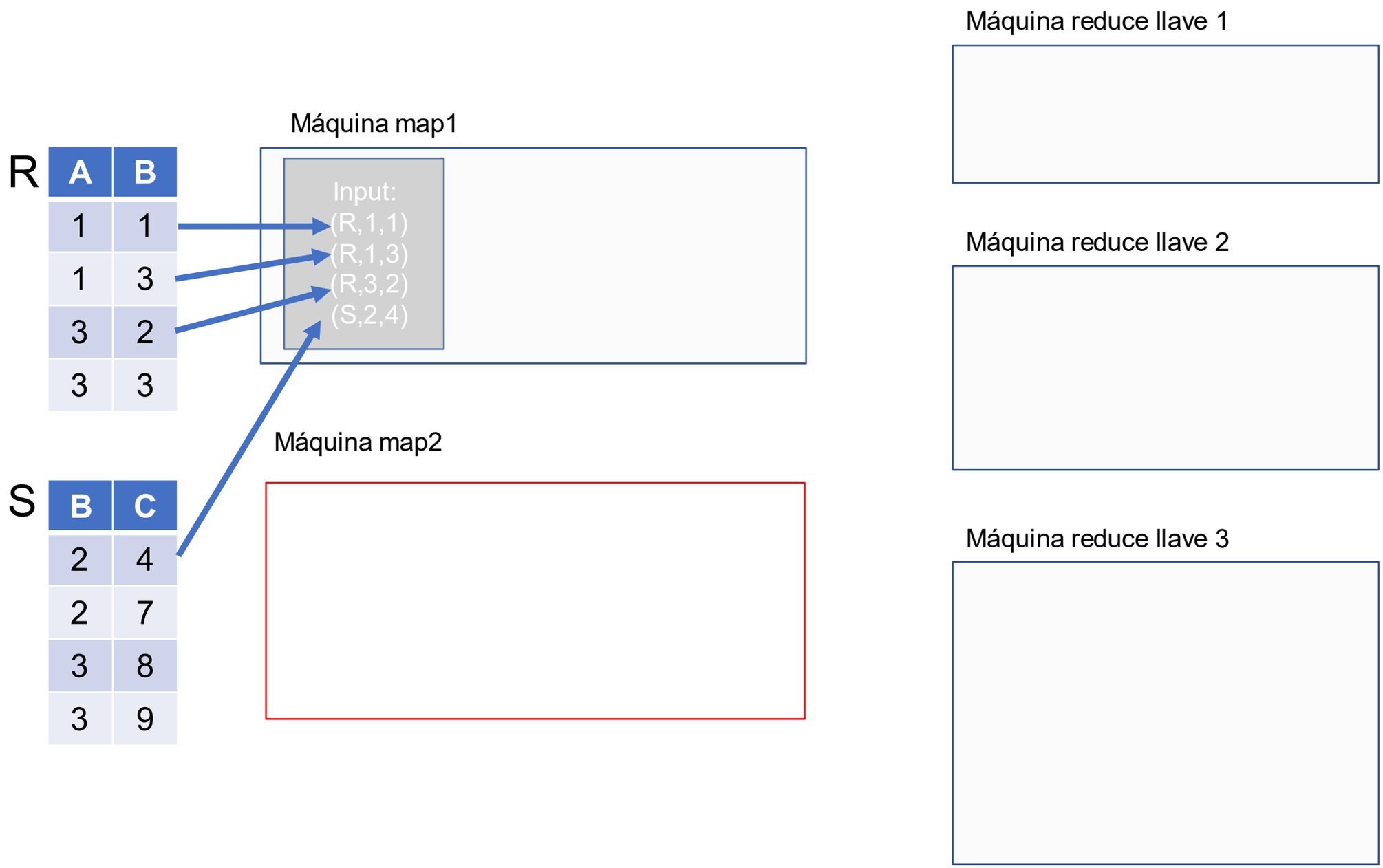
Máquina map2

Máquina reduce llave 1

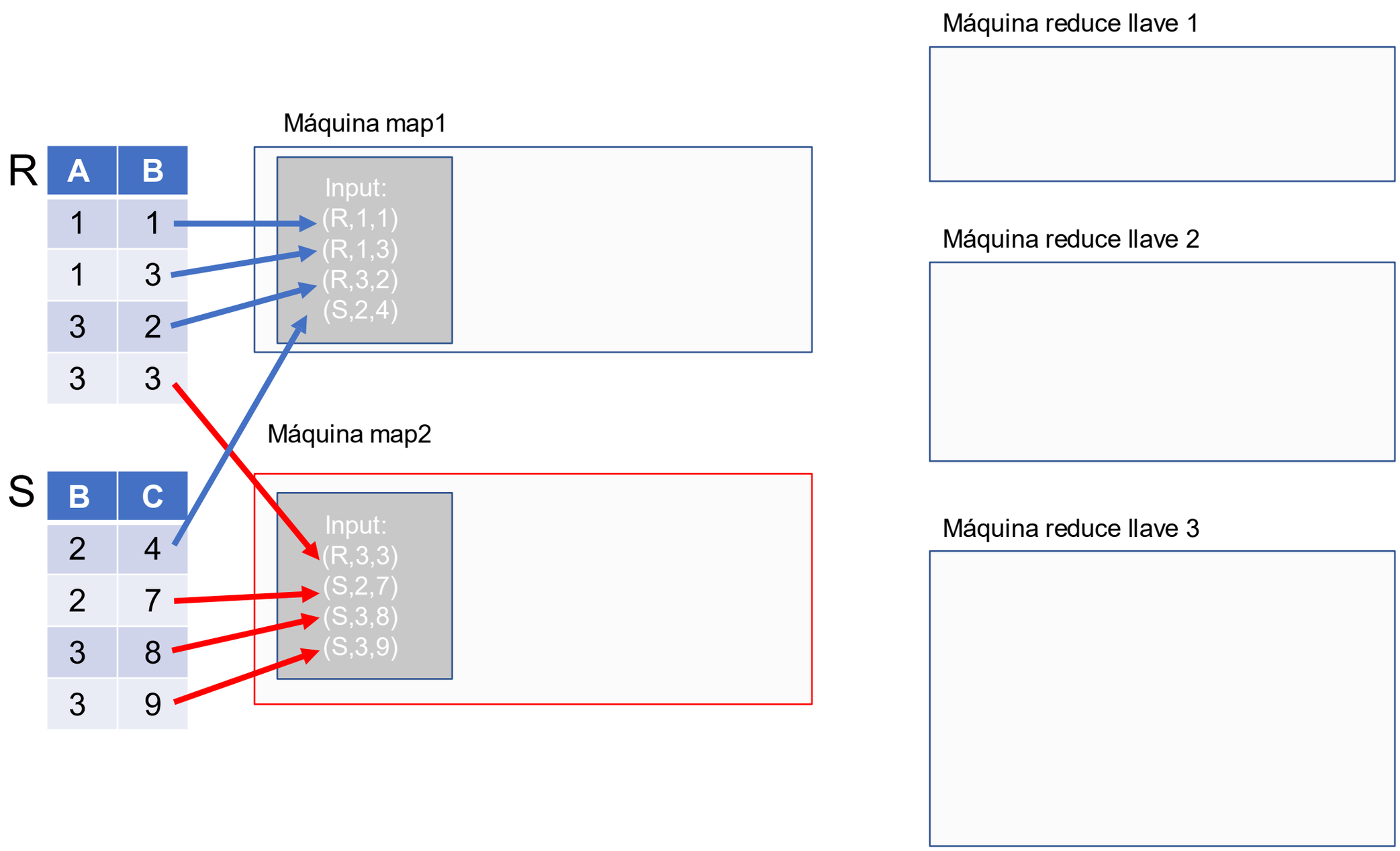
Máquina reduce llave 2

Máquina reduce llave 3

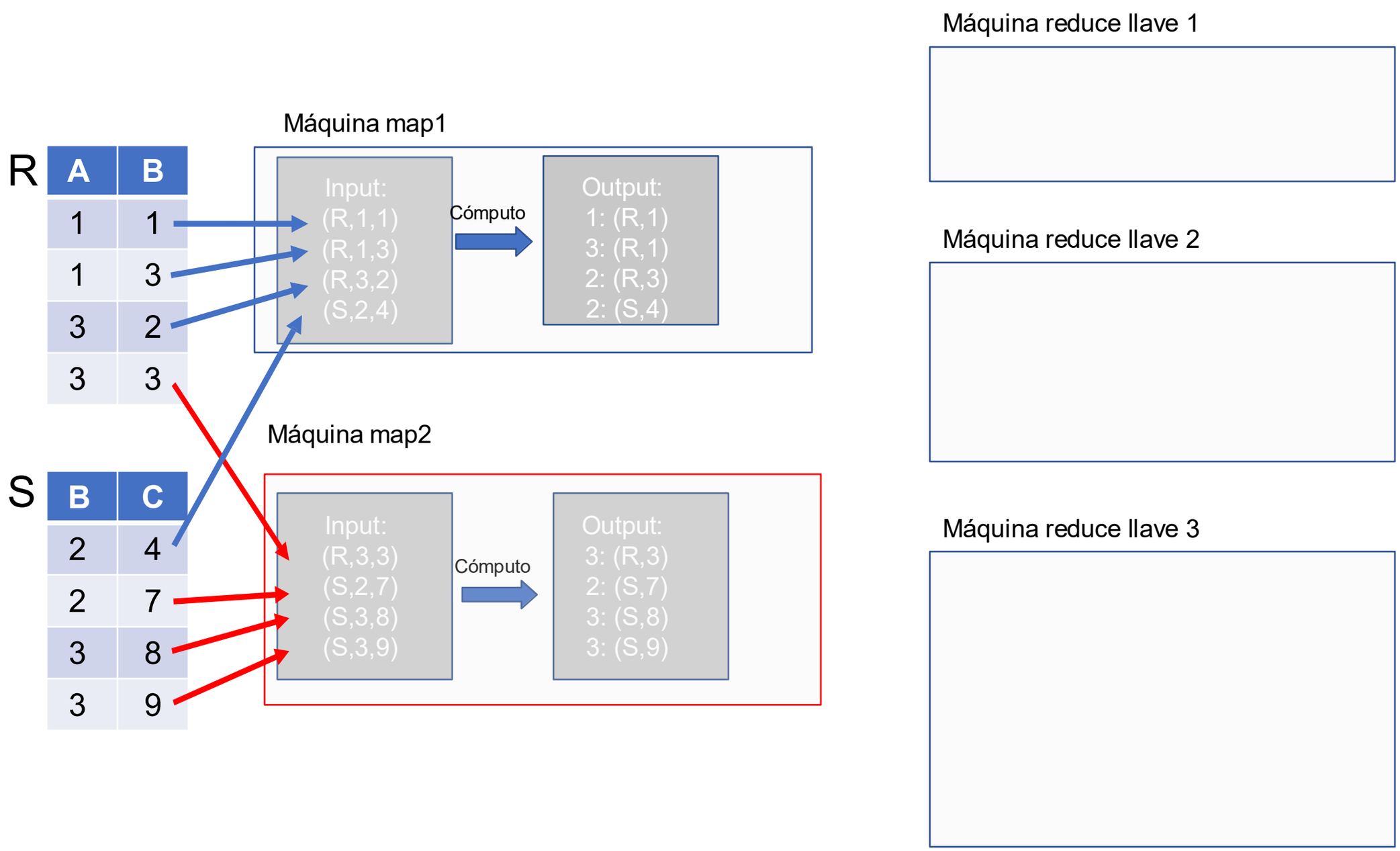
INPUT MAP


Máquina reduce llave 1
Máquina map1
R
A B
1 1
1 3
3 2
3 3
Input:
(R,1,1)
(R,1,3)
(R,3,2)
(S,2,4)
Máquina reduce llave 2
Máquina map2
S
B C
2 4
2 7
3 8
3 9
Máquina reduce llave 3

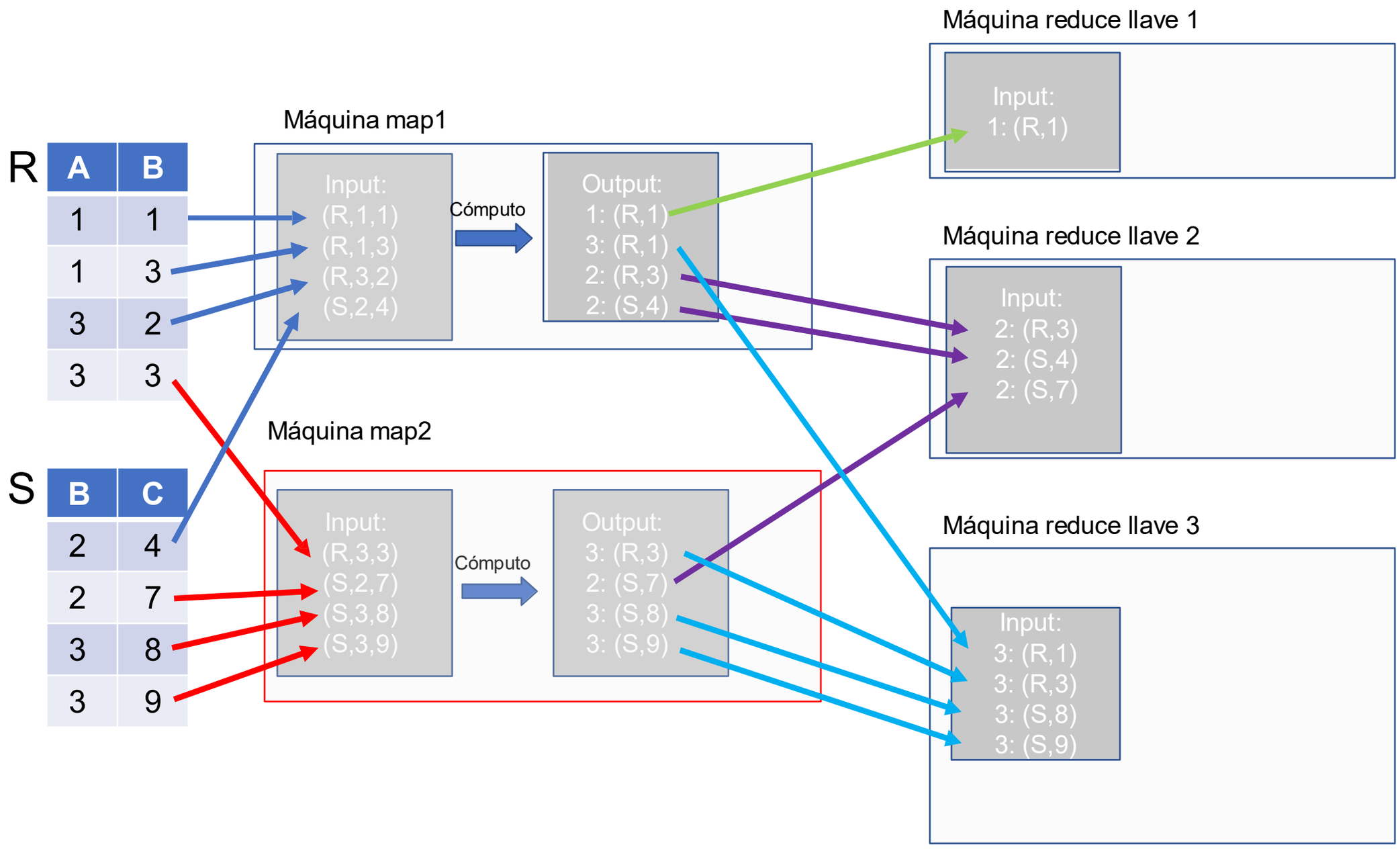
INPUT MAP

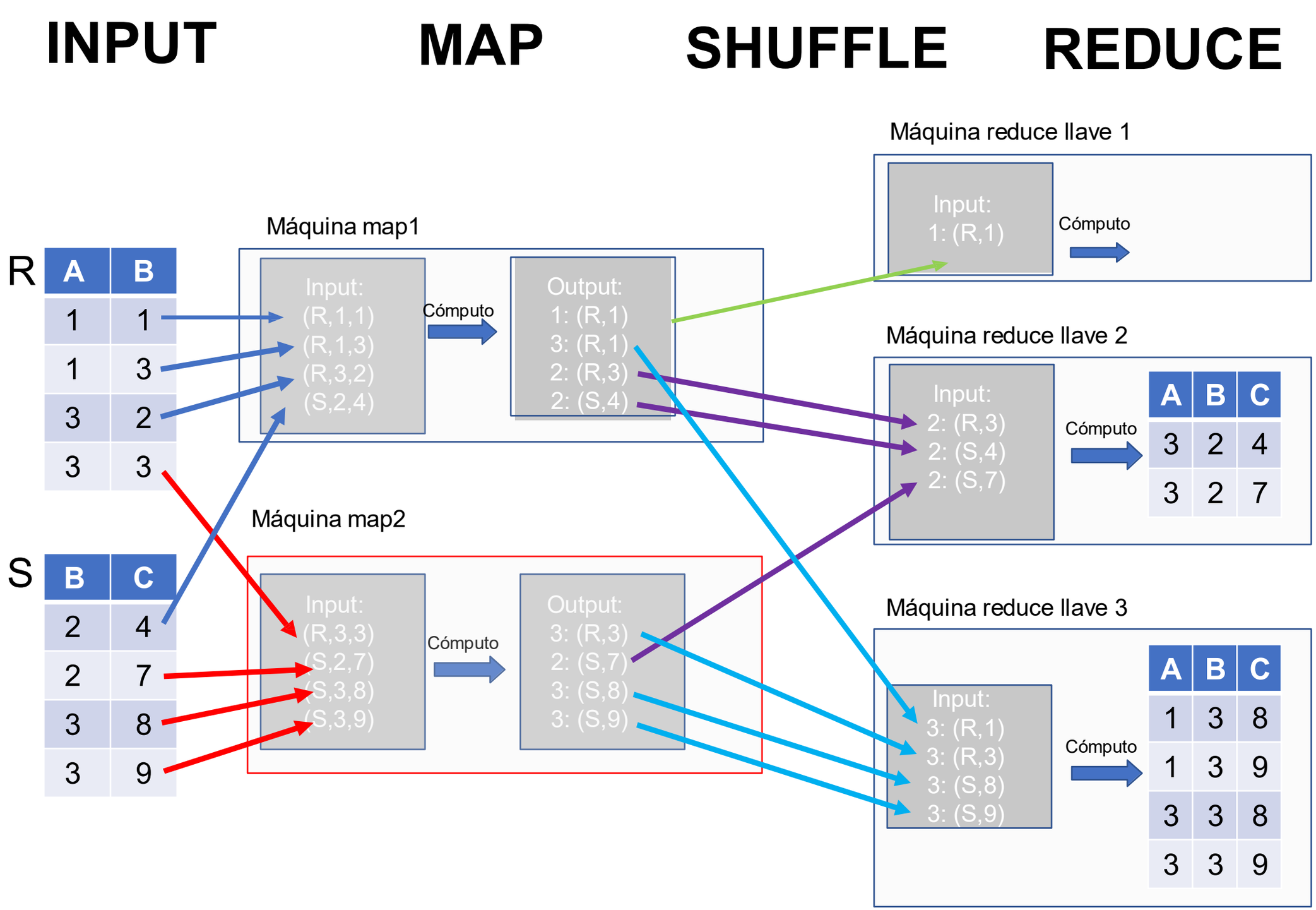

Máquina reduce llave 1
Máquina map1
R
A B
1 1
1 3
3 2
3 3
Input:
(R,1,1)
(R,1,3)
(R,3,2)
(S,2,4)
Máquina reduce llave 2
Máquina map2
S
B C
2 4
2 7
3 8
3 9
Input:
(R,3,3)
(S,2,7)
(S,3,8)
(S,3,9)
Máquina reduce llave 3

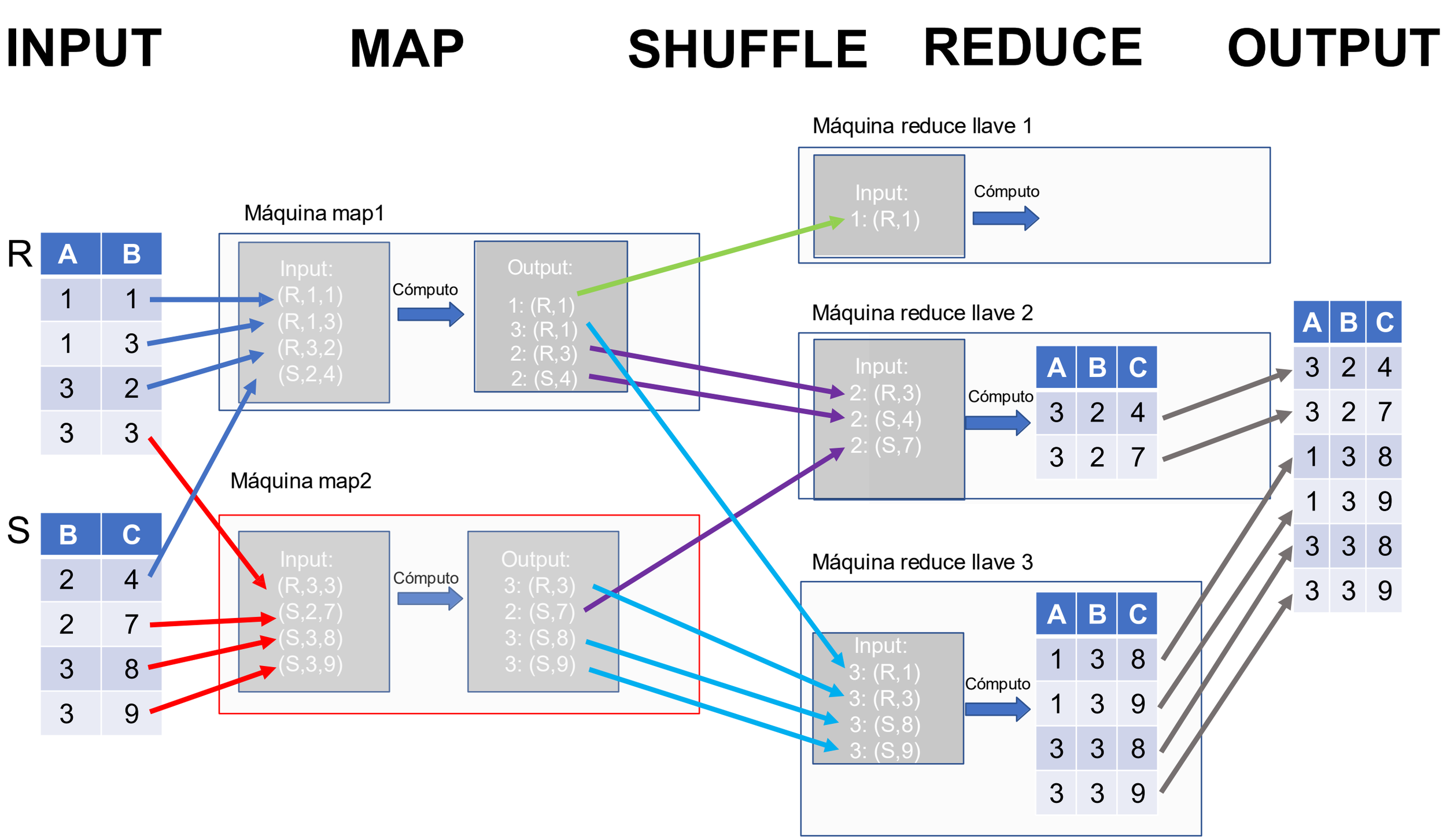
INPUT MAP


Máquina reduce llave 1
Máquina reduce llave 2
Máquina reduce llave 3
Máquina map1
Input:
(R,1,1)
(R,1,3)
(R,3,2)
(S,2,4)
Cómputo
Output:
1: (R,1)
3: (R,1)
2: (R,3)
2: (S,4)
Máquina map2
Input:
(R,3,3)
(S,2,7)
(S,3,8)
(S,3,9)
Cómputo
Output:
3: (R,3)
2: (S,7)
3: (S,8)
3: (S,9)
R
A B
1 1
1 3
3 2
3 3
S
B C
2 4
2 7
3 8
3 9

INPUT MAP SHUFFLE




INPUT
MAP
SHUFFLE
REDUCE
R
A B
1 1
1 3
3 2
3 3
S
B C
2 4
2 7
3 8
3 9
Máquina map1
Input:
(R,1,1)
(R,1,3)
(R,3,2)
(S,2,4)
Cómputo
Output:
1: (R,1)
3: (R,1)
2: (R,3)
2: (S,4)
Máquina map2
Input:
(R,3,3)
(S,2,7)
(S,3,8)
(S,3,9)
Cómputo
Output:
3: (R,3)
2: (S,7)
3: (S,8)
3: (S,9)
Máquina reduce llave 1
Input:
1: (R,1)
Cómputo
Máquina reduce llave 2
Input:
2: (R,3)
2: (S,4)
2: (S,7)
Cómputo
A B C
3 2 4
3 2 7
Máquina reduce llave 3
Input:
3: (R,1)
3: (R,3)
3: (S,8)
3: (S,9)
Cómputo
A B C
1 3 8
1 3 9
3 3 8
3 3 9


INPUT
MAP
SHUFFLE
REDUCE
OUTPUT
R
A B
1 1
1 3
3 2
3 3
S
B C
2 4
2 7
3 8
3 9
Máquina map1
Input:
(R,1,1)
(R,1,3)
(R,3,2)
(S,2,4)
Cómputo
Output:
1: (R,1)
3: (R,1)
2: (R,3)
2: (S,4)
Máquina map2
Input:
(R,3,3)
(S,2,7)
(S,3,8)
(S,3,9)
Cómputo
Output:
3: (R,3)
2: (S,7)
3: (S,8)
3: (S,9)
Máquina reduce llave 1
Input:
1: (R,1)
Cómputo
Máquina reduce llave 2
Input:
2: (R,3)
2: (S,4)
2: (S,7)
Cómputo
A B C
3 2 4
3 2 7
Máquina reduce llave 3
Input:
3: (R,1)
3: (R,3)
3: (S,8)
3: (S,9)
Cómputo
A B C
1 3 8
1 3 9
3 3 8
3 3 9
A B C
3 2 4
3 2 7
1 3 8
1 3 9
3 3 8
3 3 9

Map Reduce y BDD

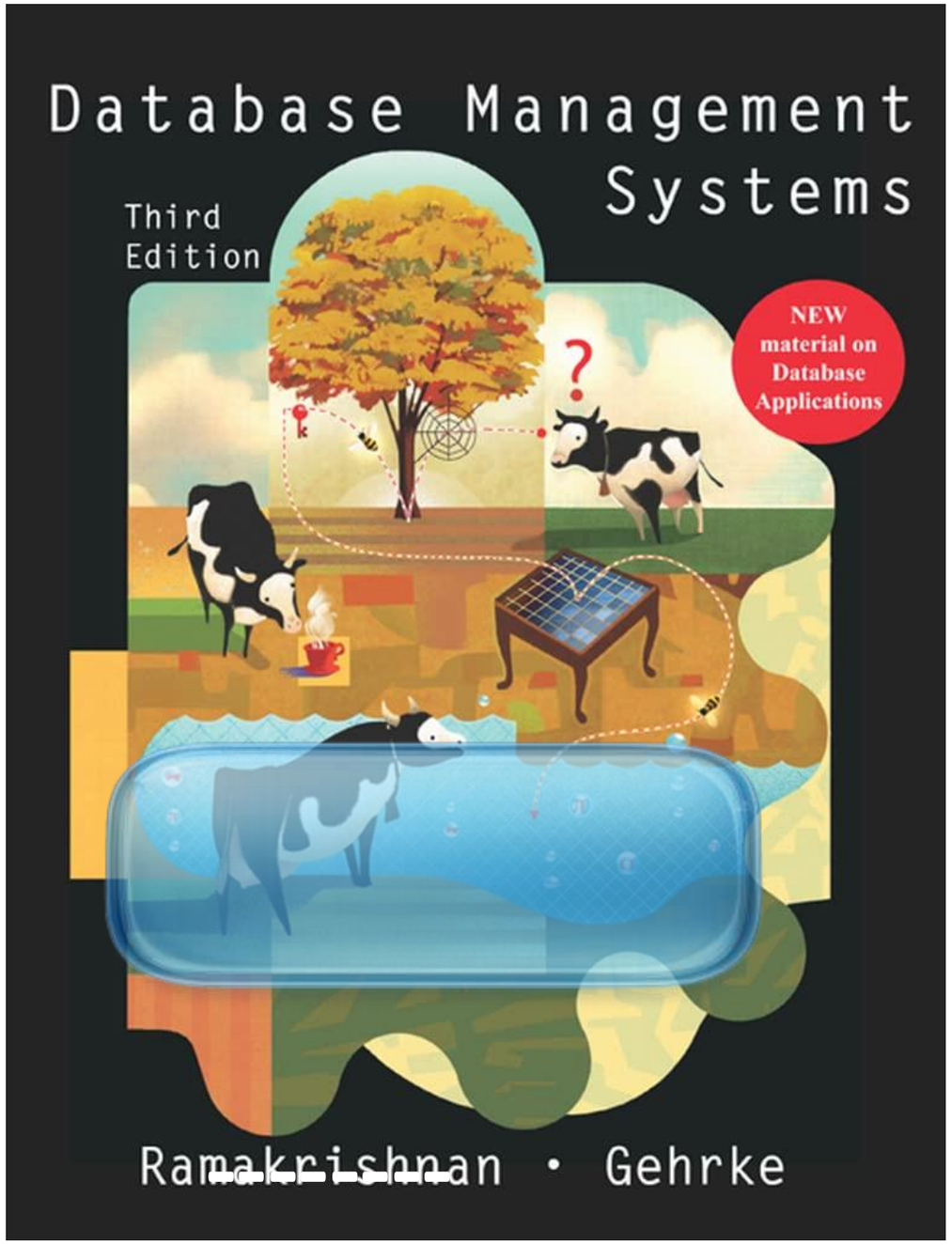
No es un descubrimiento nuevo, pero recientemente se ha visto calzar perfectamente con las necesidades de las grandes BD

Es la arquitectura más importante en sistemas que reciben grandes bases de datos

- Apache Hadoop: La implementación open source de Map -Reduce, presente en muchos sistemas con computación distribuida

Material adicional

https://www.youtube.com/watch?v=vR97-4UG7x0

https://bmdigitales-bibliotecas-uc-cl.pucdechile.idm.oclc.org/html5/DATABASE%20MANAGEMENT%20SYSTEMS/961/

NoSQL	KeyValue	Grafos	Documentos	MongoDB	Text search&retreival